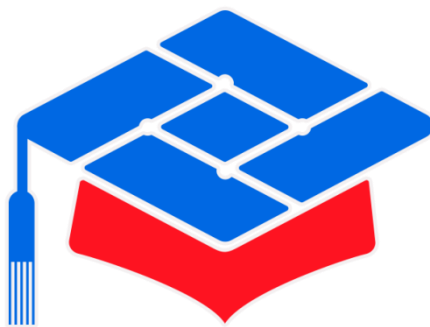




第九届

全国大学生集成电路创新创业大赛

报告类型： IP 技术报告

参赛杯赛： 龙芯中科杯

队伍编号： CICC0902564

团队名称： BZC

32 位系统安全模块 IP 技术报告

目录

32 位系统安全模块 IP 技术报告	2
1.概述	4
1.1 简介	4
1.1.1 IP 设计背景及意义	5
1.1.2 SM3 密码算法介绍:	5
1.1.3 SM4 分组密码算法介绍:	5
1.1.4 RTL 代码结构	5
1.2 设计目标	6
1.3 总体设计思路	6
1.3.1 SM3 模块:	7
1.3.4 SM4 模块:	7
1.4 功能模块设计与数据通路	8
1.4.1 SM3 模块	8
1.4.1.1 信号控制仲裁模块	8
1.4.1.2 SM3 数据填充模块	9
1.4.1.3 SM3 算法运算模块	11
1.4.2 SM4 模块	16
1.4.2.1 模式控制状态机顶层模块 SM4_TOP	16
1.4.2.2 安全密钥存储模块 key_ram_128	17
1.4.2.3 随机密钥生成模块 sm4_keygen	19
1.4.2.4 SM4 加解密执行模块	19
2.技术规格	28
2.1 功能特性与控制逻辑	28
2.1.1 SM3 模块	28
2.1.1.1 单组数据运算	28
2.1.1.2 长数据运算	29
2.1.2 SM4 模块	31
2.1.2.1 自定义密钥初始化	31
2.1.2.2 使用随机密钥生成模块进行初始化	32
2.1.2.3 使用安全密钥模块中的密钥进行初始化	33
2.1.2.4 数据加密工作模式	34
2.1.2.5 数据解密工作模式	35
2.2 性能指标	36
2.2.1 SM3 模块数据运算速率	36
2.2.1 SM4 模块数据运算速率	36
2.2.3 安全防护	37
2.3 接口定义及 APB 协议地址	37
2.3.1 APB 顶层接口列表如下	37
2.3.2 APB 顶层模块 APB 协议地址总说明列表	37
2.3.3 SM3 模块接口定义 (代码文件 sm3_top.sv)	39

2.3.4 SM4 模块接口定义（代码文件 SM4_TOP_statemac.sv）	39
2.4 资源需求.....	40
2.5.1 C 语言封装函数及软件调用方式.....	40
2.5.2 基础 APB 总线读写函数	40
2.5.3 SM3 模块功能函数.....	41
2.5.4 SM4 模块功能函数.....	43
2.5.5 中断控制处理.....	45
3.验证方案.....	47
3.1 IP 功能正确性验证.....	47
3.1.1 标准数据测试.....	47
3.1.1.1 SM3	47
3.1.1.2 SM4	47
3.1.2 足量数据测试.....	47
3.1.2.1 SM3	48
3.1.2.2 SM4	48
3.1.3 仿真测试 python 验证代码	48
3.2 FPGA 验证	49
3.2.1 SM3	49
3.2.2 SM4	49
4.测试结果.....	51
4.1 标准数据测试结果.....	51
4.1.1 SM3 结果	51
4.1.2 SM4 结果	52
4.2 足量数据测试结果.....	53
4.3 FPGA 测试结果	54
4.4 代码覆盖率报告.....	56

1.概述

1.1 简介

该 IP 为 SM3 与 SM4 算法的硬件实现，可在 50MHz 频率下稳定工作。具有完善而丰富的数据加解密功能，其中 SM3 算法 IP 模块可对小于 2^32 字长度的数据进行哈希值运算，可实现程序文件的验签功能；SM4 IP 模块具备随机密钥生成模块与安全密钥存储区，具有高安全性，同时支持多种密钥初始化模式，可快速完成大量数据文件的加解密；该 IP 涵盖需要数据加密及芯片安全运行的绝大多数使用场景；IP 具有较高处理性能，所有工作模式的数据处理速率达到 10Gbps，相比于软件实现达到专用加解密芯片优良水平。

IP 的通信协议为 APB, SM3 与 SM4 为两个独立模块挂载在 APB 顶层模块下，可方便作为安全模块集成于带 APB 总线的处理器或 SOC 中，也可作为专用加解密芯片核心工作部分。

IP 整体结构图：

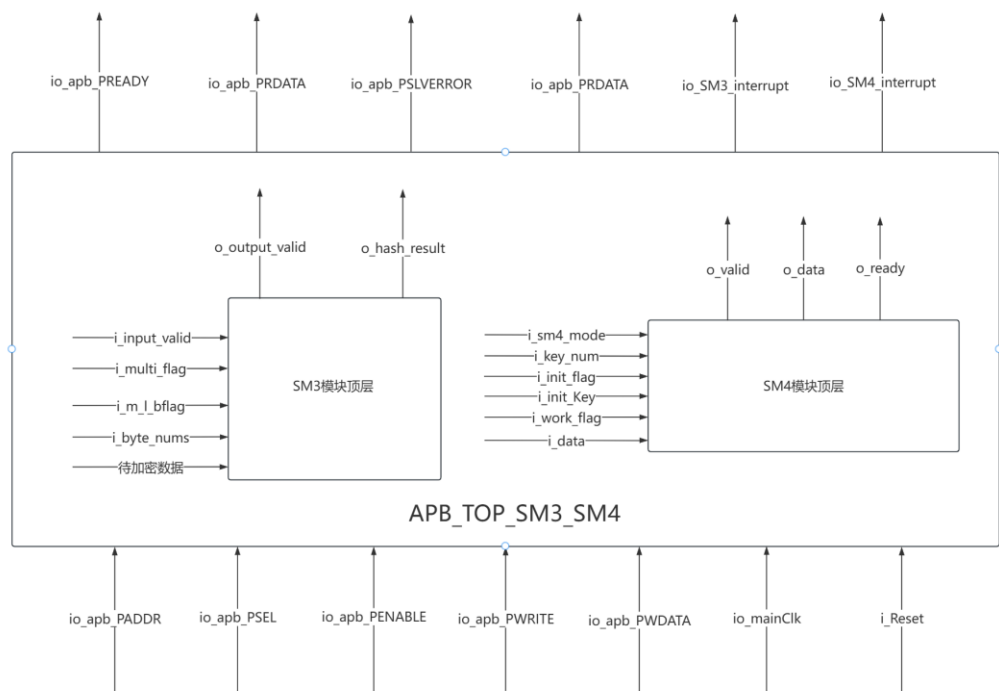


图 1-IP 整体结构图

外部 IO 信号表：

信号名称	方向	位宽	描述
io_apb_PADDR	Input	12 bits	APB 总线地址信号
io_apb_PSEL	Input	1 bit	APB 从机选择信号（高有效）
io_apb_PENABLE	Input	1 bit	APB 传输使能信号（高有效）
io_apb_PREADY	Output	1 bit	APB 从机准备好信号（高有效）
io_apb_PWRITE	Input	1 bit	APB 写使能信号（高为写，低为读）
io_apb_PWDATA	Input	32 bits	APB 写数据总线
io_apb_PRDATA	Output	32 bits	APB 读数据总线
io_apb_PSLVERROR	Output	1 bit	APB 从机错误指示信号（高有效）

io_SM3_interrupt	Output	1 bit	SM3 模块中断信号（高有效）
io_SM4_interrupt	Output	1 bit	SM4 模块中断信号（高有效）
io_mainClk	Input	1 bit	系统主时钟信号
resetCtrl_systemReset	Input	1 bit	系统复位信号（高有效）

1.1.1 IP 设计背景及意义

SM3（国密哈希算法）与 SM4（国密分组加密算法）是我国自主可控密码体系的核心算法，广泛应用于金融交易安全、政务数据交换、物联网终端身份认证及关键数据加密等信息安全场景，以及各种高保密需求的嵌入式设备应用场合。

设计高可靠、低功耗、高吞吐量的加解密 IP 核对于芯片安全模块具有重要工程价值，相较于纯软件实现，硬件 IP 核能够显著降低加解密延迟、提升数据处理吞吐量（典型场景下效率可提升 10-100 倍），同时为程序密态运行与敏感数据安全存储提供了硬件可信执行环境，是构建芯片级安全防护体系的核心基础组件。

1.1.2 SM3 密码算法介绍：

SM3 密码杂凑算法是中国商用密码杂凑算法标准。SM3 适用于商用密码应用中的数字签名和验证，是在 SHA-256 基础上改进实现的一种算法，其安全性和 SHA-256 相当，采用 Merkle-Damgard 结构。消息分组长度为 512 位，输出的摘要值长度为 256 位。

1.1.3 SM4 分组密码算法介绍：

SM4 是中国自主研发的对称加密算法（原名为 SMS4），属于分组密码体制，采用 128 位分组长度和 128 位密钥长度，输入为 128 位的秘钥和 128 位数据，加密或解密输出结果也为 128 位。其**安全性**：设计满足抗差分攻击、线性攻击等主流密码分析手段，是我国商用密码体系的重要基础算法之一。

SM4 算法在嵌入式领域主要用途如下：

1.物联网设备通信

SM4 因算法复杂度适中、硬件实现效率高，常用于物联网设备间的短数据加密（如传感器数据、设备控制指令），保障终端与服务器间的安全通信。

2. 数据存储加密

在金融、政务、医疗等领域，SM4 用于敏感数据（如用户隐私、交易记录、电子病历）的存储加密，防止数据泄露或非法访问。

1.1.4 RTL 代码结构

RTL

```
|__ SM3
|  |__ padding_code
|  |      |__ sm3_pad3.sv
|  |__ sm3_en_code
|  |      |__ Data_Extend.v
|  |      |__ SM3_Encrypt.v
|  |__ sm3_top.sv
|__ SM4
|  |__ Key_Extending.v
|  |__ key_ram128.sv
|  |__ rankey_gen.sv
|  |__ Round_Function.v
|  |__ S_Box.v
|  |__ SM4_de_entop.v
|  |__ SM4_TOP_statemac.sv
|__ APB_TOP_SM3_SM4.v
```

1.2 设计目标

总体：

- 1.SM3 与 SM4 IP 拥有较高运算速率，效率高于纯软件运算，实现硬件加速基本要求，正确率达到商用标准。
- 2.具有较高安全性，能防止数据泄露以及恶意攻击。
- 3.拥有优良时序表现，资源需求合理，可完成后端流程。

SM3 模块：

能够对不限长度数据进行哈希值运算，以在实际应用中实现对文件的验证签名功能。

SM4 模块：

- 1.支持可选择的工作模式（可使用自定义密钥或是密钥存储区特定编号密钥，加解密模式支持 ECB 和 CBC 模式，后者更具安全性）。
- 2.具有安全存储模块，用以安全存放密钥，不可被外界读取，实现保密性。
- 3.拥有随机密钥生成模块，生成的密钥只能按照自定义编号存储在密钥安全存储区中，不可被读取。
- 4.具有较高工作性能，能够快速完成对数据文件的加解密。

1.3 总体设计思路

加解密 IP 中，SM3 与 SM4 模块为两个单独的模块，共享同一个 APB 顶层，作为一个 APB 协议外设挂载在 SOC 中。软件层可以通过 APB 总线根据不同加密需要调用两个模块。

1.3.1 SM3 模块:

在按照国家标准 SM3 密码算法（《SM3 密码杂凑算法 GB/T32905—2016》）设计了 SM3 密码算法运算模块后，为实现该模块能够计算任意字节长度的数据的哈希值的功能，需要对不同长度的数据进行仲裁是否需要填充以及分组，因此我们需要设计针对非 512 比特整数倍长度的数据的填充模块，以及工作逻辑仲裁模块；

对于大于 512 比特的数据，需要外部多分组 multi_batch_flag 信号以及最后分组信号，仲裁机构将根据此两个信号决定将数据送入填充模块或是 SM3 算法运算模块。

SM3 模块结构框图如下：

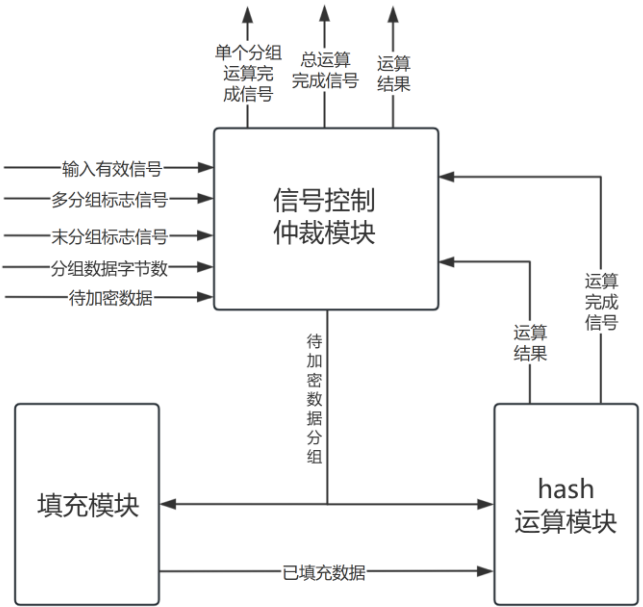


图 2-SM3 结构图

1.3.4 SM4 模块:

SM4 模块可为系统的安全运行，防止程序泄露或被窃取提供核心防护，因此要有性能较好的数据连续加密、解密功能，为提升安全性，设计了外部不可读出的密钥安全存储区以及密钥随机生成器。密钥存储区可以通过序号写入使用一个对应的密钥，也可以将该序号对应的密钥更换为新生成的随机密钥，在所有工作流程中密钥都对外不可见。另外也可以不使用密钥存储区，直接使用外部自定义的密钥进行数据加密/解密。

SM4 模块结构框图如下：

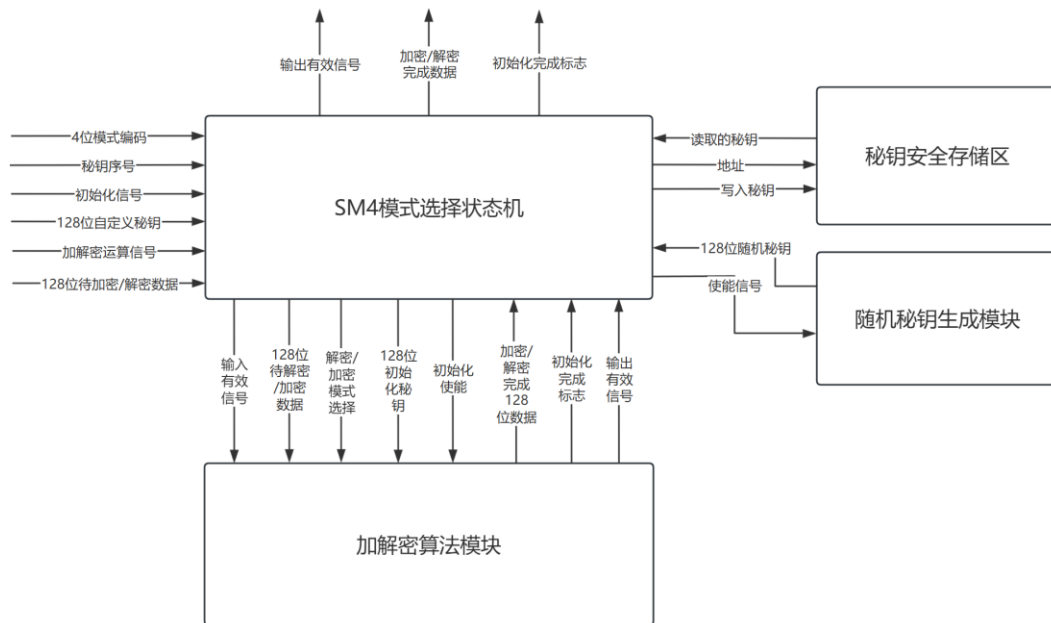


图 3-SM4 结构图

1.4 功能模块设计与数据通路

1.4.1 SM3 模块

SM3 模块由以下子模块构成：

信号控制仲裁顶层、SM3 算法运算模块，数据填充模块。

1.4.1.1 信号控制仲裁模块

该模块功能为控制输入数据信号与填充模块以及运算模块之间的交互；

该模块核心数据信号为：

1. 填充模块 sm3_padding 的输入控制信号 i_data_valid

该信号控制逻辑为：

$(i_input_valid \& \& (i_multi_flag \& i_m_l_bflag)) \mid (i_input_valid \& (!i_multi_flag))$;

当非多分组运算时，输入数据即将数据送入填充模块，若为多分组运算，只有在输入最后分组时才将数据送入填充模块，否则数据直接送入 SM3 算法运算模块

2. SM3 运算模块有效输入信号 i_input_valid

若为单分组运算，该信号由填充输出有效信号提供；

若为多分组运算非最后分组，该信号由顶层模块输入有效信号 i_input_valid 直接提供；

若为多分组运算最后分组，该信号由填充模块填充有效信号提供。

3. SM3 运算模块输入数据 i_batch_data0

该接口输入逻辑与有效输入信号 i_input_valid 相同。

1.4.1.2 SM3 数据填充模块

模块接口定义：

信号名称	类型	位宽	功能描述
i_clk	输入	1-bit	时钟信号，模块同步操作的基准时钟。
i_rst	输入	1-bit	复位信号（高有效）
i_data_in	输入	512-bit	待填充的原始数据输入
i_data_valid_len	输入	10-bit	输入数据的有效长度
i_data_valid	输入	1-bit	输入数据有效标志（高有效）
i_m_l_bflag	输入	1-bit	最后分组标志信号
i_bytes_multi	输入	32-bit	长数据运算模式下最后分组输入时传入总数据字节数
o_padded_data0	输出	512-bit	填充结果的高 512 位数据
o_padded_data1	输出	512-bit	填充后的低 512 位数据（仅当输入有效长度超过 448bit 时输出，用于多块填充场景）。
o_pad_done	输出	1-bit	高 512bits 数据填充完成标志（高有效）
o_pad_done1	输出	1-bit	低 512bits 数据填充完成标志（高有效）

该模块完成的工作是对单组数据填充为 512bits 整数倍后输出给 SM3 运算模块，该模块严格按照国家 SM3 密码算法标准设计，完成的填充工作表述如下：

填充步骤

- 第一步：添加比特“1”和若干“0”
对于长度为 l 比特的消息 m ，先在其末尾添加比特“1”，然后再添加 k 个“0”。这里的 k 是满足 $l+1+k \equiv 448 \bmod 512$ 的最小非负整数。
- 第二步：添加 64 位比特串
在经过第一步填充后的消息后面再添加一个 64 位比特串，该比特串是长度 l 的二进制表示。
- 第三步：得到填充后的消息 m'
经过上述填充操作后，消息 m' 的比特长度为 512 的倍数

该模块采用状态机设计，将填充分为三个周期完成，工作流程图如下：

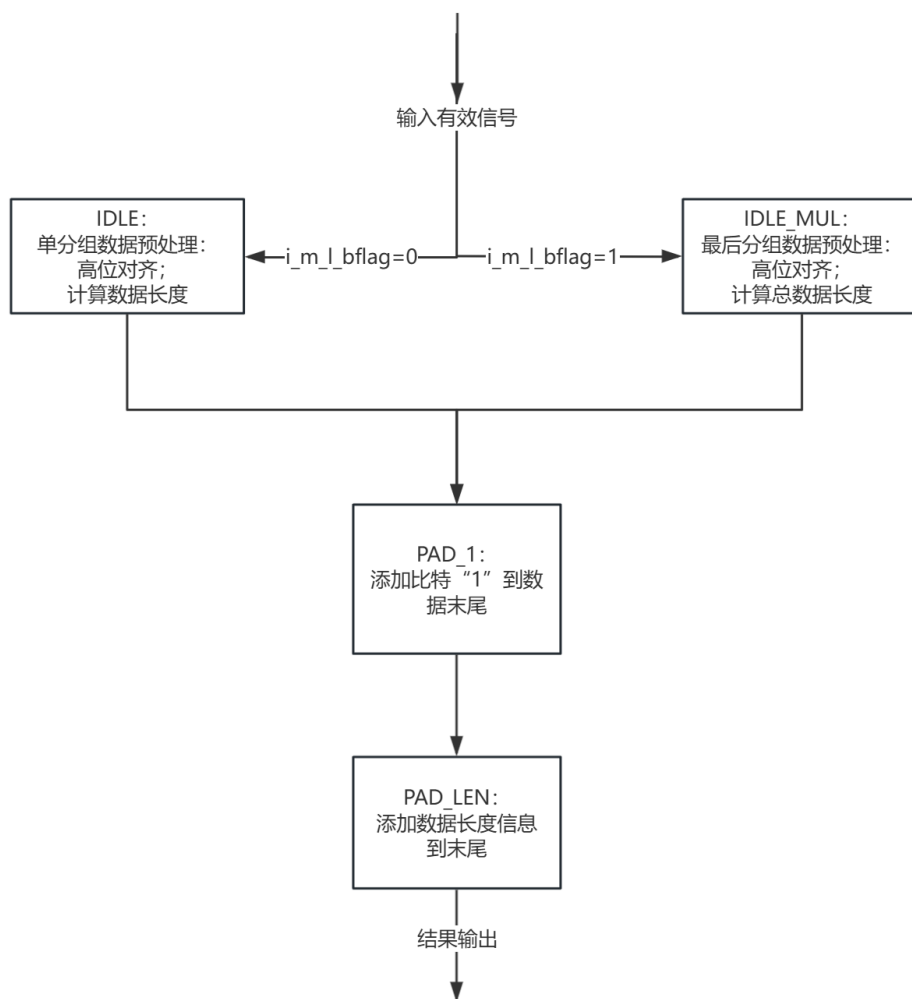


图 4-填充模块流程图

该模块各状态工作内容说明：

IDLE0:

空闲状态，复位所有信号后无操作；

IDLE:

单分组运算数据预处理，若多分组最后分组有效信号未拉高，将从 IDLE 进入此状态。字节长度数据左移 5 为，即乘以 32，成为比特长度，将数据高位对到 1024 位寄存器 padded_data，以兼容填充后数据大于 512 比特的情况；

IDLE_MUL:

多分组运算的最后分组数据数据预处理，运算内容与 IDLE 一致，但数据长度计算使用的是 i_bytes_multi 信号数据；

PAD_1:

将比特信号“1”添加到高位对齐后的输入数据末尾；

PAD_LEN:

判断数据比特长度是否小于 448，若是，则将数据比特长度信号添加到 padded_data 寄存器高 512 位的末尾，仅拉高 o_pad_done 信号，结束填充并返回空闲状态。若大于等于 448 位，则添加到低 512 位末尾，并将 o_pad_done 与 o_pad_done1 信号拉高。

1.4.1.3 SM3 算法运算模块

模块执行算法数学描述

根据《SM3 密码杂凑算法 GB/T32905—2016》国家标准文档，该模块需要实现以下运算流程：

该流程称为迭代压缩，其数学表达如下：

设填充后的消息为 m' ，将其按 512 比特为一组进行划分，得到分组序列：

$$B^{(0)}B^{(1)} \dots B^{(n-1)}$$

其中分组数 n 由公式 $n = (l + k + 65)/512$ 计算得出。（ l 为原始消息长度， k 为填充的 0 比特数）

随后，设定初始 256 比特中间值 V^0 。通过循环迭代（次数为从 $i = 0$ 到 $i = n - 1$ ），每次迭代中利用压缩函数 CF 对当前中间值 V^i 和消息分组 B^i 进行处理，计算得到下一个中间值，数学描述为：

$$\begin{aligned} &FOR\ i = 0\ to\ n - 1 \\ &\quad V^{i+1} = CF(V^i, B^i) \\ &ENDFOR \end{aligned} \quad (公式\ 1-4-1)$$

经过 n 次迭代后，最终的迭代压缩结果为 V^n ；
完成上述运算后，输出结果的为：

$$y = V^n \quad (公式\ 1-4-2)$$

CF 函数为处理步骤中关键的压缩函数，其内容描述如下：

令 A, B, C, D, E, F, G, H 为字寄存器， $SS1, SS2, TT1, TT2$ 为中间变量，

$$CF(V^i, B^i): \quad (公式\ 1-4-3)$$

BEGIN

$$ABCDEFGH \leftarrow V(i)$$

FOR $j = 0$ TO 63

$$SS1 \leftarrow ((A \ll 12) + E + (T_j \ll j)) \ll 7$$

```

SS2  $\leftarrow$  SS1  $\oplus$  (A  $\ll$  12)
TT1  $\leftarrow$  FFj(A, B, C) + D + SS2 + Wj'
TT2  $\leftarrow$  GGj(E, F, G) + H + SS1 + Wj
D  $\leftarrow$  C
C  $\leftarrow$  B  $\ll$  9
B  $\leftarrow$  A
A  $\leftarrow$  TT1
H  $\leftarrow$  G
G  $\leftarrow$  F  $\ll$  19
F  $\leftarrow$  E
E  $\leftarrow$  P0(TT2)
ENDFOR
Vi+1  $\leftarrow$  ABCDEFGH  $\oplus$  Vi

```

END

其中，字的存储为大端格式。

上述流程中嵌套调用的 FFj, GGj 为布尔函数，P0, P1 为置换函数，定义如下：

FFj(X, Y, Z): 公式 1-4-4

$$\begin{aligned} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z) & 16 \leq j \leq 63 \end{aligned}$$

GGj(X, Y, Z): 公式 1-4-5

$$\begin{aligned} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\neg X \wedge Z) & 16 \leq j \leq 63 \end{aligned}$$

P0, P1: 公式 1-4-6

$$\begin{aligned} P0(X) &= X \oplus (X \ll 9) \oplus (X \ll 17) \\ P1(X) &= X \oplus (X \ll 15) \oplus (X \ll 23) \end{aligned}$$

式中 X 为字。

压缩函数中 W 为将消息分组 Bⁱ 按以下方法扩展生成 132 个字 W₀, W₁, ..., W₆₇, W₀' W₁' ..., W₆₃' 用于压缩函数 CF:

(1) 将消息分组 Bⁱ 划分为 16 个字 W₀, W₁, ... W₁₅

(2) FOR j = 16 TO 67 公式 1-4-7

$$W_j \leftarrow P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \ll 15)) \oplus (W_{j-13} \ll 7) \oplus W_{j-6})$$

ENDFOR

(3) FOR j = 0 TO 63 公式 1-4-8

$$W_j' = W_j \oplus W_{j+4}$$

ENDFOR

完成上述运算过程的 SM3 运算模块分为 sm3_encrypt 与 Data_Extend 两个子模块，顶层模块为 sm3_encrypt，在其接收到数据及数据有效信号后，首先将数据传入 Data_Extend 消息拓展模块；

该模块主要采用组合逻辑完成 132 个字的拓展。

硬件实现设计

Data_Extend 子模块

首先将 512 位填充后消息数据寄存器暂存后调整字节顺序（大端转小端），赋值给中间寄存器数组 w_padding_data_f；

相关代码如下：

```
always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst) begin
        ri_padding_data <= 'd0;
        ri_padding_valid <= 'd0;
        ri_padding_valid_ld <= 'd0;
    end else begin
        ri_padding_data <= i_padding_data ;
        ri_padding_valid <= i_padding_valid;
        ri_padding_valid_ld <= ri_padding_valid;
    end
end

genvar g_i;
generate
for(g_i = 0 ; g_i < 16 ; g_i = g_i + 1)
begin:Encode
    assign w_padding_data_f[g_i*32 +31 : g_i*32] = ri_padding_data[511 - g_i*32 : 511 - 31 - g_i*32];
end
endgenerate
```

第二级处理：

将 512 位填充数据直接分割为 16 个 32 位初始字($W_0 - W_{15}$)，赋值给中间寄存器数组 ro_extend_data；

相关代码：

```
genvar g_exi;
generate
for(g_exi = 0 ; g_exi < 16 ; g_exi = g_exi + 1)
begin:Extend_Word1
    always@(*)
    begin
        if(i_rst)
            ro_extend_data[g_exi] <= 'd0;
        else
            ro_extend_data[g_exi] <= w_padding_data_f[(g_exi * 32) + 31:g_exi * 32];
        end
    end
endgenerate
```

第三级运算：

通过函数定义的方式实现 SM3 标准中的 $P1$ 函数
并通过 generate 语句重复调用递推生成 W_{16} 至 W_{67} ；

相关代码：

```

genvar g_exj;
generate
)   for(g_exj = 16 ; g_exj < 68 ; g_exj = g_exj + 1)
    begin:Extend_Word2
        assign w_P1_X[g_exj] = ro_extend_data[g_exj-16] ^ ro_extend_data[g_exj-9] ^ {ro_extend_data[g_exj-3][16:0],ro_extend_data[g_exj-3][31:17]};
        assign w_P1[g_exj] = P1(w_P1_X[g_exj]);
        assign w_word2_mid1[g_exj] = {ro_extend_data[g_exj - 13][24:0],ro_extend_data[g_exj - 13][31:25]};
    )
    always@(*)
    begin
    )
        if(i_rst)
            ro_extend_data[g_exj] <= 'd0;
        else
            ro_extend_data[g_exj] <= w_P1[g_exj] ^
                                    w_word2_mid1[g_exj]^
                                    ro_extend_data[g_exj - 6];
    )
    )
    end
)
end
endgenerate

```

第四级运算：通过异或运算生成 $W_{68}-W_{131}$,

相关代码：

```

genvar g_exk;
generate
)   for(g_exk = 0 ; g_exk < 64 ; g_exk = g_exk + 1)
    begin:Extend_Word3
    )
        always@(*)
    )
        begin
    )
            if(i_rst)
                ro_extend_data[g_exk + 68] <= 'd0;
            else
                ro_extend_data[g_exk + 68] <= ro_extend_data[g_exk] ^ ro_extend_data[g_exk + 4];
    )
        end
    )
    end
)
endgenerate

```

时序控制与输出部分：

该部分使拓展字通过寄存器 `ro_extend_data_1d` 打拍，确保与时钟同步；有效信号 `ro_extend_valid` 延迟一拍输出，与数据对齐；最终将 132 个 32 位字拼接为 4224 位的 `o_extend_data` 输出。

相关代码：

```

genvar g_ddi;
generate
    for(g_ddi = 0 ; g_ddi < 132 ; g_ddi = g_ddi + 1)
    begin:reg_1d
        always@(posedge i_clk,posedge i_rst)
        begin
            if(i_rst)
                ro_extend_data_1d[g_ddi] <= 'd0;
            else if(ri_padding_valid)
                ro_extend_data_1d[g_ddi] <= ro_extend_data[g_ddi];
            else
                ro_extend_data_1d[g_ddi] <= ro_extend_data_1d[g_ddi];
        end
    end
endgenerate

always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst)
        ro_extend_valid <= 'd0;
    else
        ro_extend_valid <= ri_padding_valid_1d;
end

```

sm3_encrypt 模块

该模块将拓展后的 132 个字进行迭代压缩，为简化代码编写，将重复执行的步骤定义为函数以便调用。

相关部分代码：

```
function [255:0] iterator;
    input [255 :0] i_V ;
    input [31 :0] i_j ;
    input [31 :0] w_Tj_y ;
    input [31 :0] w_Wj ;
    input [31 :0] w_Wj_ ;

    reg [31:0] w_ss1 ;
    reg [31:0] w_ss2 ;

    function [31:0] P0;
        input [31 :0] X;
    begin
        P0 = X ^ {X[22:0],X[31:23]} ^ {X[14:0],X[31:15]};
    end
endfunction

function [31:0] FFj;
    input [31 :0] X;
    input [31 :0] Y;
    input [31 :0] Z;
    input [31 :0] j;
begin
    if(j<16)
        FFj = X ^ Y ^ Z;
    else
        FFj = (X & Y) | (X & Z) | (Y & Z);
    end
endfunction

function [31:0] GGj;
    input [31 :0] X;
    input [31 :0] Y;
    input [31 :0] Z;
    input [31 :0] j;
begin
```

迭代过程编写：对于单组 512 比特消息，需要进行 64 次 iterator 函数的迭代运算，若消息为多分组，总运算次数为分组数×64，为保证时序性能，将 64 次迭代运算设计为时序逻辑，通过 64 组标志信号 cmp 与 tmp，在单个时钟周期内行完一次 iterator 函数后将本次迭代完成信号 cmp 以及下一迭代开始标志信号 tmp 拉高。

结果输出：最后一次迭代完成，tmp64 被拉高后，将所有 cmp 及 tmp 信号置零等待下一数据输入，并进行杂凑值计算

实现代码：

```
always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst) begin
        ro_Encrypt_Data <= 'd0;
    end else if(cmp64&!tmp64) begin
        ro_Encrypt_Data <= r_iterator_V[64] ^ r_iterator_V[0];
    end else
        ro_Encrypt_Data<=ro_Encrypt_Data;
end
```

单消息分组模式下，r_iterator_V[0]的值为 SM3 标准默认值：
7380166f4914b2b9172442d7da8a0600_a96f30bc163138aae38dee4db0fb0e4e；

当运行模式为多分组时 r_iterator_V[0]为上一消息分组运算杂凑值结果，其控制逻辑为：

```
always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst)
        r_iterator_V[0] <= 256'h7380166f4914b2b9172442d7da8a0600_a96f30bc163138aae38dee4db0fb0e4e;
    else if(!r_multi_flag)
        r_iterator_V[0] <= 256'h7380166f4914b2b9172442d7da8a0600_a96f30bc163138aae38dee4db0fb0e4e;
    else if(r_multi_flag&& r_batch_cnt>1) begin//cnt>0 after the second data in, this code works then always work
        r_iterator_V[0] <= r_V_multi_d3;
    end
end
```

r_V_multi_d3 为上一消息分组打拍寄存结果，当多分组运算非第一分组时，才会使用该值作为 r_iterator_V[0]。

完成该步骤后，将结果与输出有效信号一起输出，完成单次 SM3 算法运算。

1.4.2 SM4 模块

SM4 模块由以下子模块构成：

模式控制状态机顶层，随机密钥生成模块，密钥安全存储区，SM4 算法实现模块。

1.4.2.1 模式控制状态机顶层模块 SM4_TOP

该模块采用状态机设计，根据输入模式控制信号 i_sm4_mode 进入不同状态，操控其他子模块的数据输入和工作，并将结果输出。

该模块状态机各状态工作内容如下：

idle： 空闲状态，将所有工作信号复位

gen_key： 进入随机密钥生成模式，模式选择信号 i_sm4_mode 为 0001。拉高密钥生成使能信号和计数器使能信号，计数到生成模块完成密钥生成所需周期后，使能 SM4 模块的初始化信号并将密钥写入密钥存储区。

decrypt： 数据解密模式，若模式选择信号为 0100，将输入的数据送入 SM4 模块并拉高解密使能输入信号。

encrypt 加密模式，模式选择信号为 1000

fetch_key 获取密钥安全存储区的特定编号密钥并进行 SM4 模块初始化。模式选择信号为 0010。

init_key 使用外部密钥初始化 SM4 模块。

init_IV 选择 CBC 加解密模式时初始化通过随机密钥生成模块生成随机的 128 位 IV 值，并写入密钥存储区对应模块。

1.4.2.2 安全密钥存储模块 key_ram_128

该模块是设计用于安全存储 128 位密钥的专用存储模块，采用分布式 RAM 实现(*ram_style = "distributed" *)，支持 128 个存储条目（7 位地址线，2⁷=1282⁷=128）。模块通过掩码混淆、诱饵地址防护、写锁定机制等安全设计，防止密钥被非法读取或篡改，适用于高安全等级的密码学场景。
该模块接口定义为：

信号名称	方向	位宽	功能描述
i_clk	输入	1 位	系统时钟信号
i_rst_n	输入	1 位	异步复位信号（低电平有效），用于初始化模块状态及存储内容。
i_ren	输入	1 位	读使能信号（高电平有效），允许从指定地址读取密钥数据。
i_wen	输入	1 位	写使能信号（高电平有效），允许向指定地址写入密钥数据。
i_addr	输入	7 位	读写操作的目标编号（地址），范围 0~127（共 128 个存储单元，其中 32、95、127 为防数据嗅探伪地址陷阱）。
i_wdata	输入	128 位	写入 RAM 的 128 位密钥数据。
o_key_out	输出	128 位	从 RAM 中读取的 128 位密钥数据（经掩码混淆后的值）。
o_valid	输出	1 位	读操作有效标志（高电平有效），表示 o_key_out 数据可信。
o_write_ack	输出	1 位	写操作确认信号（高电平有效），表示写操作成功完成。

掩码混淆实现机制：

```
wire [127:0] mask = MASKING ? {64{2'b10}} : 128'h0; //可通过初始配置MASKING参数修改掩码位
```

读取时真实数据与掩码异或后输出：

$$o_key_out \leq ram[i_addr] \wedge mask;$$

当 MASKING=1 时，读取的密钥数据会与固定模式的掩码（{64{2'b10}}）异或，混淆真实值；MASKING=0 时输出原始数据。

功能：通过异或掩码干扰侧信道攻击，防止密钥真实值被直接探测。

诱饵地址防护实现机制：

```

if (!i_rst_n) begin
    if (ADDR_DECOY > 0) begin
        ram[32]   <= 128'hDEADBEEF_DEADBEEF_DEADBEEF_DEADBEEF;
        ram[95]   <= 128'hCAFEBABE_CAFEBABE_CAFEBABE_CAFEBABE;
        ram[127]  <= 128'hBADFOOD_BADFOOD_BADFOOD_BADFOOD;
    end
end

```

复位时初始化诱饵地址（ADDR_DECOY>0 时生效）

```

if (i_addr < 128) begin
    if ((ADDR_DECOY > 0) && ((i_addr == 7'd32) || (i_addr == 7'd95) || (i_addr == 7'd127))) begin
        o_key_out <= 128'h0;
        o_valid <= 1'b0;
    end
end

```

读操作时，若访问诱饵地址则返回 0 并标记无效。

复位时，地址 32、95、127（由 ADDR_DECOY=3 指定）被预填充为固定假数据；若非法访问这些地址，模块返回 0 且标记无效，掩盖真实密钥位置。

该机制通过预填充假数据的“诱饵地址”，误导攻击者对真实密钥存储位置的猜测。

写锁定机制：该机制限制已写入地址的二次修改，防止密钥被恶意覆盖。

实现代码：

```

if (i_wen) begin
    if (i_addr < 128) begin
        if ((ADDR_DECOY > 0 && ((i_addr == 7'd32) || (i_addr == 7'd95) || (i_addr == 7'd127))) || (write_lock_mask[i_addr])) begin
            o_write_ack <= 1'b0;
        end
        else begin
            ram[i_addr] <= i_wdata;
            o_write_ack <= 1'b1;
            if (WRITE_LOCK) begin
                write_lock_mask[i_addr] <= 1'b1;
            end
        end
    end
end
end

```

写入有效地址后，write_lock_mask 对应位被置 1，后续对该地址的写操作将被拒绝（o_write_ack=0），避免密钥被篡改。

复位与初始化：

功能：异步复位时清空 RAM、初始化安全参数，确保模块启动状态可控。

附加防护设计

模块通过 ram_shield 信号组（4 个固定值的 128 位常量）进一步增强抗侧信道能力，这些信号不参与实际存储，但可干扰功耗分析等侧信道攻击。

相关代码：

```
(* keep = "true" *) wire [127:0] ram_shield [0:3];
assign ram_shield[0] = 128'hFFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFF;
assign ram_shield[1] = 128'h00000000_00000000_00000000_00000000;
assign ram_shield[2] = 128'hAAAAAAAA_AAAAAAAAA_AAAAAAAAA_AAAAAAAAA;
assign ram_shield[3] = 128'h55555555_55555555_55555555_55555555;
```

1.4.2.3 随机密钥生成模块 sm4_keygen

该模块是 SM4 密码算法中用于生成 128 位随机密钥的专用模块，基于线性反馈移位寄存器（LFSR）实现，通过本原多项式驱动的移位操作生成高随机性的密钥序列，适用于 SM4 算法的密钥初始化场景。

该模块信号接口：

信号名称	方向	位宽	功能描述
i_clk	输入	1 位	系统时钟信号，模块同步逻辑的触发源
i_rst_n	输入	1 位	异步低电平复位信号
i_keygen_en	输入	1 位	密钥生成使能信号（高电平有效）：高电平：触发 LFSR 移位并输出新密钥；低电平：停止移位，保持当前状态。
o_key	输出	128 位	128 位 SM4 密钥输出信号，直接映射 LFSR 当前状态（仅在 i_keygen_en=1 时更新）。

该模块工作逻辑：

模块核心为 128 位 LFSR（lfsr_state，128 位宽寄存器），通过本原多项式驱动移位形成输出随机密钥。

本原多项式代码定义为：

```
wire feedback = lfsr_state[127] ^ lfsr_state[125] ^ lfsr_state[100] ^ lfsr_state[98];
```

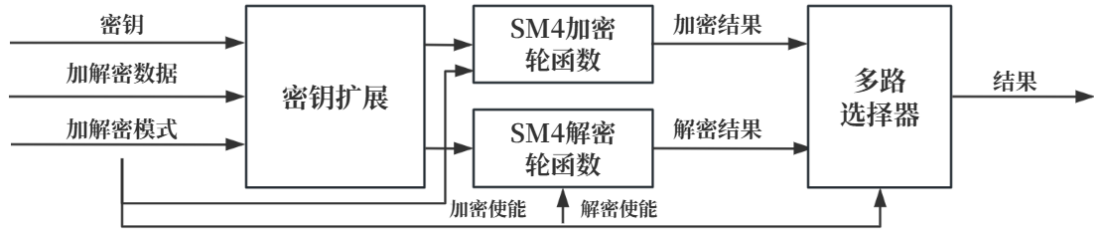
```
LFSR 移位操作：lfsr_state <= {lfsr_state[126:0], feedback};
```

设计特点

- 1.高随机性：该模块，本原多项式驱动的 LFSR 使序列周期最长为 $(2^{128}-1)$ ，避免重复模式；
- 2.抗预测性：通过非全零初始状态和多位异或反馈逻辑，提高了生成密钥的不可预测性；
- 3.低资源消耗：基于移位寄存器和组合逻辑实现，适合硬件加速场景；
- 4.该模块通过本原多项式 LFSR 生成 128 位随机密钥兼顾随机性与硬件实现效率，满足 SM4 算法对密钥随机性的要求。

1.4.2.4 SM4 加解密执行模块

该部分顶层模块为 SM4_de_entop，其子模块为 Round_Function、S_Box 与 Key_Extending；总体数据通路及工作流程：



模块执行的算法数学描述

根据《SM4 分组密码算法 GB/T32907—2016》国家标准文档，该模块需要实现以下运算流程：

1. 加密算法

SM4 加密算法由 32 次迭代运算和 1 次反序变换 R 组成：

设明文输入为 128 位连续 4 个字组合 (X_0, X_1, X_2, X_3) ，密文输出为同样 128 位 4 个字 (Y_0, Y_1, Y_2, Y_3) ，轮密钥为 $rk_i, (i = 0, 1, 2, \dots, 31)$ ；

加密算法的运算过程如下：

a) 32 次迭代运算：

$$X_{i+4} = F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i), i = 0, 1, \dots, 31 \quad \text{公式 1-4-9}$$

b) 反序变换：

$$(Y_0, Y_1, Y_2, Y_3) = R(X_{32}, X_{33}, X_{34}, X_{35}) = (X_{35}, X_{34}, X_{33}, X_{32}) \quad \text{公式 1-4-10}$$

2. 解密算法

解密变换与加密变换结构相同，唯一的区别在于轮密钥的使用顺序。在进行解密操作时，需要使用轮密钥顺序为 $(rk_{31}, rk_{30}, \dots, rk_0)$

上述算法中 F 函数为轮函数，定义为：

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk) \quad \text{公式 1-4-11}$$

式中：

$\oplus$ 表示异或运算， T 是一个特定的变换函数：

合成置换 T ：

T 是一个可逆变换，它由非线性变换 τ 和线性变换 L 复合而成，即 $T(.) = L(\tau(.))$

非线性变换 τ :

设输入为 $A = (a_0, a_1, a_2, a_3)$ ，输出为 $B = (b_0, b_1, b_2, b_3)$ ，它们满足以下关系式:

$$(b_0, b_1, b_2, b_3) = \tau(A) = (Sbox(a_0), Sbox(a_1), Sbox(a_2), Sbox(a_3)) \quad \text{公式 1-4-12}$$

Sbox 数据见下表:

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
A	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
B	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
C	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
D	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
E	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
F	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

表-1 Sbox 数据

硬件实现设计

S_Box 模块

该模块为 SM4 算法中 S 盒的硬件实现。

S 盒体现了 SM4 分组密码算法的非线性变换。

我们将 S 盒看作是一个二维的数组，通过查找表的方式实现一个 S 盒。由于 S 盒参数是固定的，可以通过启动复位信号实现初始化。主要代码如下：

```

always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst) begin
        r_s_Box[0] = 128'hD690E9FECCE13DB716B614C228FB2C05;
        r_s_Box[1] = 128'h2B679A762ABE04C3AA44132649860699;
        r_s_Box[2] = 128'h9C4250F491EF987A33540B43EDCFAC62;
        r_s_Box[3] = 128'hE4B31CA9C908E89580DF94FA758F3FA6;
        r_s_Box[4] = 128'h4707A7FCF37317BA83593C19E6854FA8;
        r_s_Box[5] = 128'h686B81B27164DA8BF8EB0F4B70569D35;
        r_s_Box[6] = 128'h1E240E5E6358D1A225227C3B01217887;
        r_s_Box[7] = 128'hD40046579FD327524C3602E7A0C4C89E;
        r_s_Box[8] = 128'hEABF8AD240C738B5A3F7F2CEF96115A1;
        r_s_Box[9] = 128'hE0AE5DA49B341A55AD933230F58CB1E3;
        r_s_Box[10] = 128'h1DF6E22E8266CA60C02923AB0D534E6F;
        r_s_Box[11] = 128'hD5DB3745DEFD8E2F03FF6A726D6C5B51;
        r_s_Box[12] = 128'h8D1BAF92BDDBC7F11D95C411F105AD8;
        r_s_Box[13] = 128'h0AC13188A5CD7BBD2D74D012B8E5B4B0;
        r_s_Box[14] = 128'h8969974A0C96777E65B9F109C56EC684;
        r_s_Box[15] = 128'h18F07DEC3ADC4D2079EE5F3ED7CB3948;
    end
end

always@(posedge i_clk,posedge i_rst)
begin
    if(i_rst)
        ro_s_data <= 'd0;
    else case(w_Y)
        0      :ro_s_data <= r_s_Box[w_X][127:120];
        1      :ro_s_data <= r_s_Box[w_X][119:112];
        2      :ro_s_data <= r_s_Box[w_X][111:104];
        3      :ro_s_data <= r_s_Box[w_X][103: 96];
        4      :ro_s_data <= r_s_Box[w_X][95 : 88];
        5      :ro_s_data <= r_s_Box[w_X][87 : 80];
        6      :ro_s_data <= r_s_Box[w_X][79 : 72];
        7      :ro_s_data <= r_s_Box[w_X][71 : 64];
        8      :ro_s_data <= r_s_Box[w_X][63 : 56];
        9      :ro_s_data <= r_s_Box[w_X][55 : 48];
        10     :ro_s_data <= r_s_Box[w_X][47 : 40];
        11     :ro_s_data <= r_s_Box[w_X][39 : 32];
        12     :ro_s_data <= r_s_Box[w_X][31 : 24];
        13     :ro_s_data <= r_s_Box[w_X][23 : 16];
        14     :ro_s_data <= r_s_Box[w_X][15 :  8];
        15     :ro_s_data <= r_s_Box[w_X][7  :  0];
    endcase
end

```

Key_Extending 模块

该模块实现了 SM4 算法的密钥拓展步骤:

密钥扩展函数需要对给定的密钥扩展 32 次形成 32 个轮密钥。32 个轮密钥通过分别作用于加解密过程中的 32 次迭代。

对于密钥扩展函数，主要是通过一系列的异或和 S 盒替换实现。因此，对于单个的密钥扩展函数，我们通过异或、移位等组合逻辑实现并插入 3 个寄存器形成 3 级流水线。

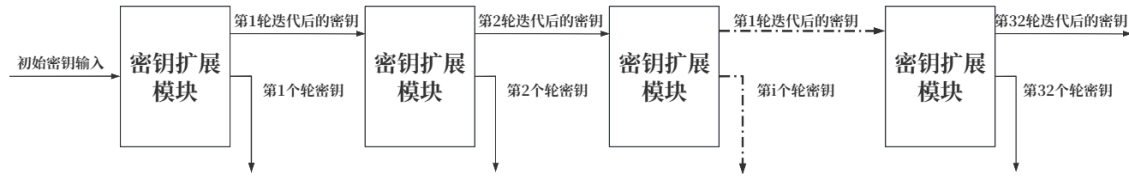


图 6-秘钥拓展实现流程图

为了便于扩展 32 次形成 32 个轮密钥，我们将单个密钥扩展模块的输出插入一级寄存器，然后将上一次密钥迭代后输出的结果输出给下一级密钥扩展模块，形成串级密钥扩展模块。同时，在每一轮迭代完成后都输出对应的轮密钥供加密轮函数模块或解密轮函数模块使用。

因为密钥扩展需要 32 次迭代，每一次迭代都需要上一次的密钥 K_i , K_{i+1} , K_{i+2} , K_{i+3} , 所以我们使用 generate for 语句同时例化 32 个密钥扩展模块，同时将上一个模块迭代后的密钥作为下一个模块的输入，如下图所示。

```
genvar i;
generate
  for(i = 0 ; i < 32 ; i = i + 1)
    begin:Gen_Key
      SM4_key_extend Key_Extend_ux(
        .clk_in          (clk_in          ),
        .rst_in          (i_rst | initial_valid_in ),
        .i_in            (i              ),
        .initial_key_in   (reg_Initial_key[i]   ),
        .initial_valid_in (reg_Initial_valid[i] ),
        .encrypt_key_out  (w_Encrypt_Key[i]    ),
        .encrypt_valid_out(w_Encrypt_valid[i]   ),
        .K_out            (w_K[i]             )
      );

      always@(posedge clk_in,posedgerst_in)
      begin
        if(i_rst)
          reg_rK[i] <= 'd0;
        else
          reg_rK[i] <= w_Encrypt_Key[i];
      end
      if(i > 0) begin
        always@(posedge clk_in,posedgerst_in)
        begin
          if(i_rst) begin
            reg_Initial_key[i]   <= 'd0;
            reg_Initial_valid[i] <= 'd0;
          end else if(initial_valid_in) begin
            reg_Initial_key[i]   <= 'd0;
            reg_Initial_valid[i] <= 'd0;
          end else begin
            reg_Initial_key[i]   <= w_K[i - 1];
            reg_Initial_valid[i] <= w_Encrypt_valid[i - 1] ;
          end
        end
      end
    end
  endgenerate
```

Round_Function 模块

该模块为 SM4 算法中轮函数的硬件实现。

加密过程和解密过程都需要用到轮函数，对于给定的明文，需要迭代 32 次形成新的密文。最后取第 29 次、第 30 次、第 31 次、第 32 次输出的密文的高字通过反序变换作为加解密的结果作为输出。

轮函数主要通过一系列的异或和 S 盒替换实现。因此，对于单个的轮函数迭代，我们也通过异或、移位等组合逻辑实现并插入 3 个寄存器形成 3 级流水线。

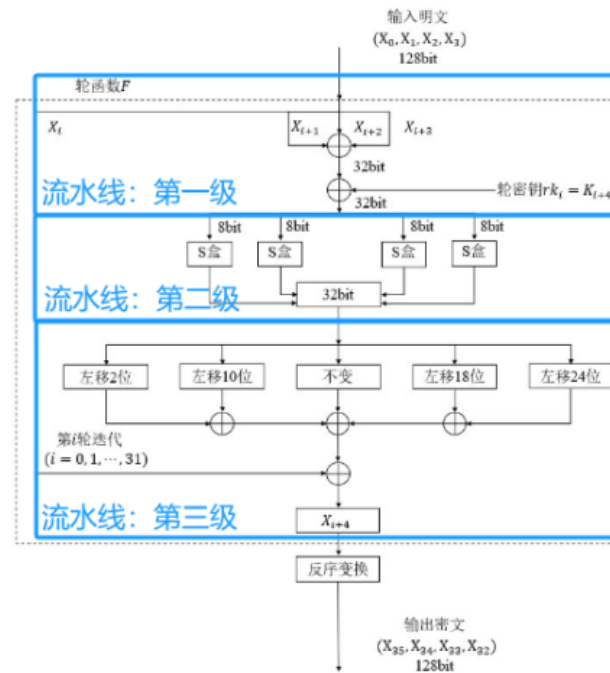


图 7 轮函数实现模块结构图

其中，加解密过程也是通过异或、移位、S 盒替换的组合逻辑实现，组合逻辑插入 3 个寄存器形成 3 级流水线。

```
always@(posedge clk_in,posedge rst_in)begin
    if(rst_in)
        ro_data <= 32'd0;
    else
        ro_data <= reg_Ki_result; //每一次迭代生成的Xi+4
    end

    always@(posedge clk_in,posedge rst_in)begin
        if(rst_in)
            ro_next_data <= 128'd0;
        else
            ro_next_data <= {ri_data_3d[95:0],reg_Ki_result}; //每一次迭代生成的新的密文
        end
    end
```

如上图所示，加密或解密的轮函数需要有两个输出，第一个是明文经过第 i 次加密或解密后的 128 位结果，准备作为第 $i+1$ 次的输入。第二个是输出的 X_{i+4} ，对于 SM4 加密或者解密，本质就是将第 29 次，30 次，31 次，32 次得到的 X_{i+4} 拼接作为最终结果的输出。

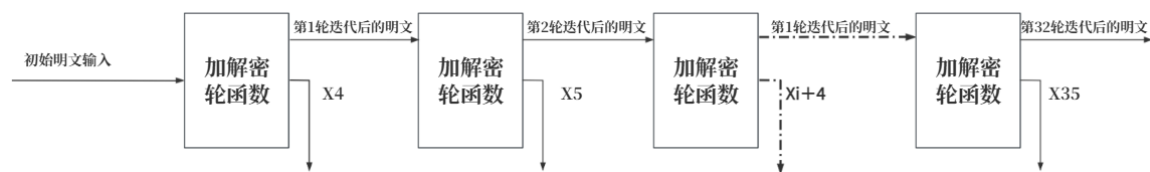


图 8-轮函数实现流程

在例化时，因为加密和解密使用的轮函数顺序不同，所以我们使用 `generate for` 语句分别例化了 32 个加密用的轮函数和 32 个解密用的轮函数，如下图所示。

```
genvar j;
generate
  for(j = 0;j < 32;j = j + 1)
    begin:Gen_Round
      SM4_round_function round_function_encrypt(
        .clk_in          (clk_in          ),
        .rst_in          (i_rst          ),
        .i_in            (j              ),
        .rk_in           (reg_rK[j]       ),
        .data_in          (reg_round_data_encrypt[j] ),
        .valid_in         ((reg_round_valid_encrypt[j]&&(i_axis_mode==2'b01))),
        .data_out         (wire_round_data_encrypt[j] ),
        .valid_out        (wire_round_valid_encrypt[j] ),
        .next_data_out    (wire_round_next_data_encrypt[j] )
      );
      SM4_round_function round_function_decrypt(
        .clk_in          (clk_in          ),
        .rst_in          (i_rst          ),
        .i_in            (j              ),
        .rk_in           (reg_rK[31-j]    ),
        .data_in          (reg_round_data_decrypt[j] ),
        .valid_in         ((reg_round_valid_decrypt[j]&&(i_axis_mode==2'b10))),
        .data_out         (wire_round_data_decrypt[j] ),
        .valid_out        (wire_round_valid_decrypt[j] ),
        .next_data_out    (wire_round_next_data_decrypt[j] )
      );
    end;
endgenerate
```

SM4_de_entop 顶层模块

该顶层模块通过例化 `Round_Function` 与 `Key_Extending` 子模块完成数据的加解密并将结果输出。接口定义如下：

端口名称	方向	位宽(bits)	类型
i_clk	输入	1	时钟信号
i_rst	输入	1	复位信号（高有效）
i_init_Key	输入	128	初始化密钥数据
i_init_valid	输入	1	初始化密钥有效信号
i_mode	输入	2	模式控制信号，01 为加密，10 为解密
i_data	输入	128	待处理数据
i_valid_top	输入	1	输入数据有效信号
o_ready	输出	1	模块初始化完成的就绪信号
o_data	输出	128	处理结果数据
o_valid	输出	1	结果有效信号

加密与解密工作流程分别如下：

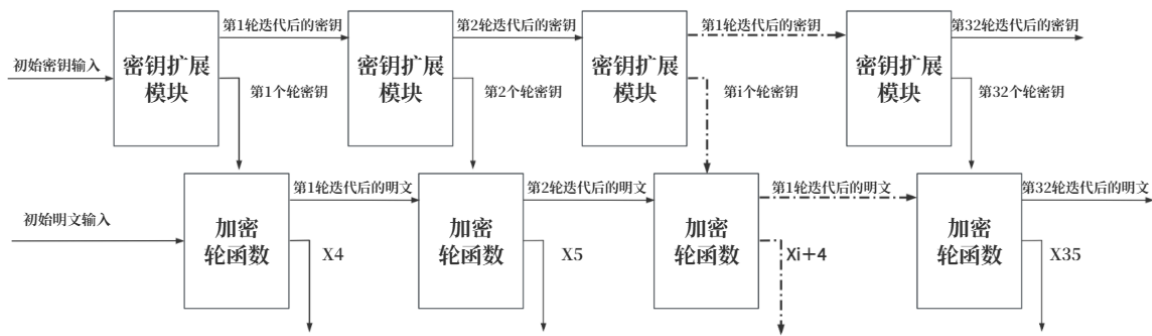


图 9-SM4 加密流程图：

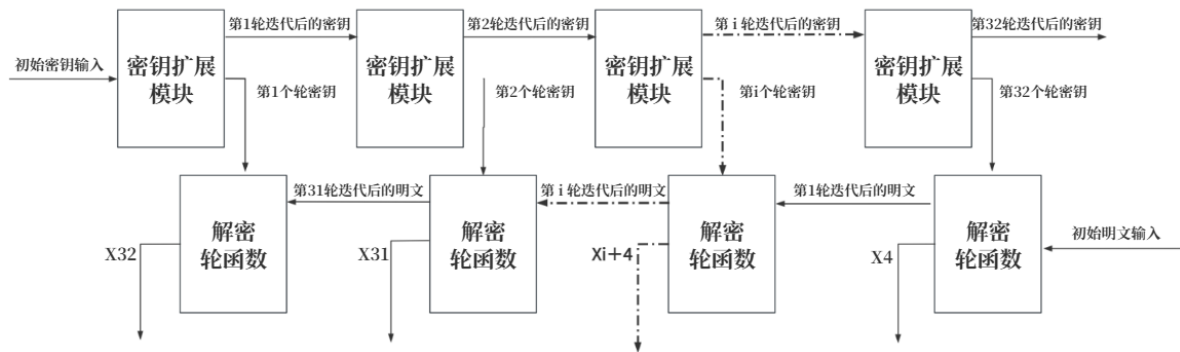


图 10-SM4 解密流程图

SM4 分组密码算法的加密过程和解密过程的实现非常类似，区别在于使用的轮密钥的顺序不同。我们让加密过程和解密过程共用一个密钥扩展模块，可以有效节省资源，减小面积。

2.技术规格

2.1 功能特性与控制逻辑

IP 通过 APB 协议与外界进行数据交互，SM3 与 SM4 为独立的使用不同 APB 地址的工作模块。APB 顶层具有两个中断输出分别对应 SM3 与 SM4 模块的结果输出有效信号，当某个模块完成数据加解密运算后将输出中断信号，便于系统进行处理。

2.1.1 SM3 模块

该模块可以实现对任意以字节为单位长度的数据的 SM3 算法运算，当待运算数据长度小于等于 16 字节(512bits)时，同时拉高输入有效信号并传送数据，等待 o_output_valid 信号拉高即加密完成可接收结果输出。
单次运算所需时钟周期数为 143cycles。

当数据长度大 16 字节(512bits)，需要将数据按 16 字节为单位进行分组（最后一个分组可以不满 16 字节），依次按大端顺序（高位在前）传输每个分组的数据，这种工作模式下需要保持 i_multi_flag 拉高，并且在输入最后一个分组时将 i_m_l_bflag 信号拉高，等待最后一个分组完成运算后在将两个信号置零。

详细工作时序如下：

2.1.1.1 单组数据运算

输入参数：

消息数据	i_data	512 bits
字节长度	i_byte_nums	6 bits

输出：

消息摘要	o_hash_result	256 bits
------	---------------	----------

时序图如下：

i_data 与 i_input_valid 信号同时跳转，同时输入字节长度信息 i_byte_nums。
等待 o_output_valid 信号拉高后可读取数据结果输出 o_hash_result 。

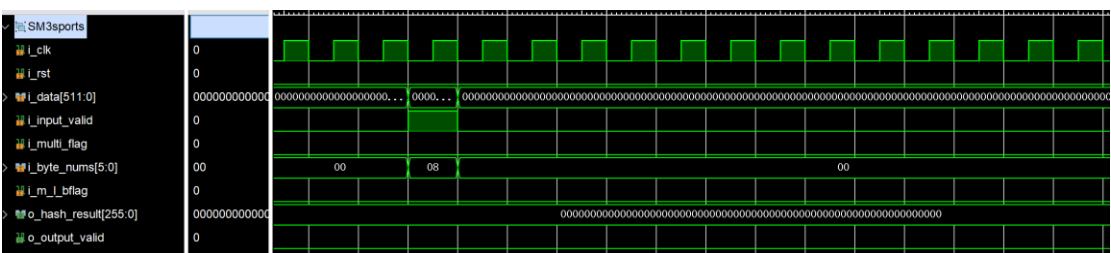


图 11-SM3 模块单组数据运算输入时序

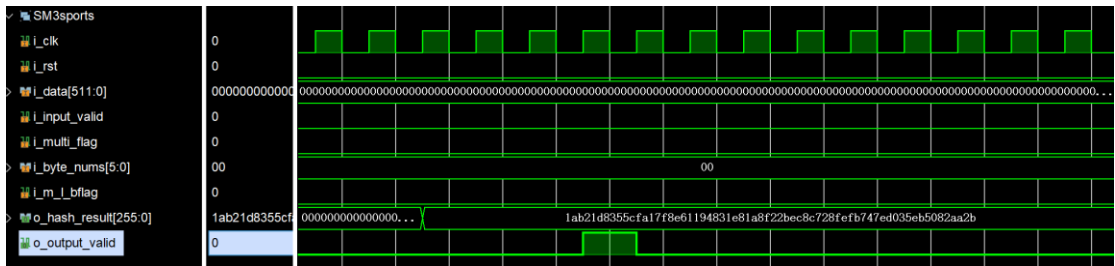


图 12-SM3 模块单组数据运算结果输出时序

APB 顶层写地址及数据顺序：

写入顺序	地址(12 位十六进制)	数据
1	204	i_data[511:480]
2	208	i_data[479:448]
3	20C	i_data[447:416]
4	210	i_data[415:384]
5	214	i_data[383:352]
6	218	i_data[351:320]
7	21C	i_data[319:288]
8	220	i_data[287:256]
9	224	i_data[255:224]
10	228	i_data[223:192]
11	22C	i_data[191:160]
12	230	i_data[159:128]
13	234	i_data[127:96]
14	238	i_data[95:64]
15	23C	i_data[63:32]
16	240	i_data[31:0]
17	244	6 位字节数(<16)i_byte_nums
18	200	32'b001

结果读取 APB 地址：

读取顺序	地址(12 位十六进制)	数据
1	248	o_hash_result[31: 0]
2	24C	o_hash_result[63: 32]
3	250	o_hash_result[95: 64]
4	254	o_hash_result[127: 96]
5	258	o_hash_result[159:128]
6	25C	o_hash_result[191:160]
7	260	o_hash_result[223:192]
8	264	o_hash_result[255:224]

2.1.1.2 长数据运算

输入参数：

消息数据 i_data 512 bits
字节长度 i_byte_nums 6 bits

输出:

消息摘要 o_hash_result 256 bits

将消息按 512 比特为单位分组，从高位开始依次输入，等待当前数据分组运算完成后输入下一数据分组，保持 i_multi_flag 信号为高，输入最后数据分组时须将 i_m_l_bflag 信号拉高，全部分组运算完成后才可將 i_multi_flag 与 i_m_l_bflag 信号置零。

运算最后数据分组前的数据时序图如下:

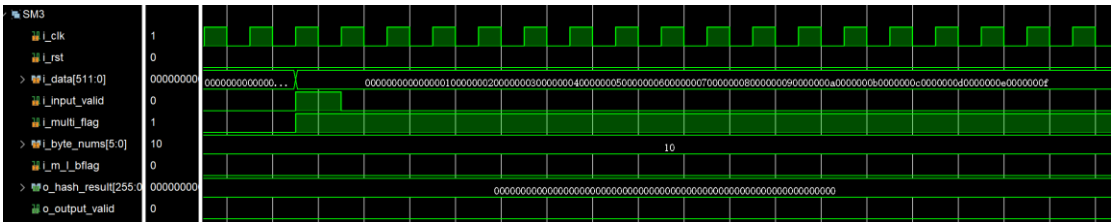


图 13-SM3 模块多组数据运算非最后分组输入时序

等待 o_output_valid 信号拉高后输入下一数据分组进行运算:

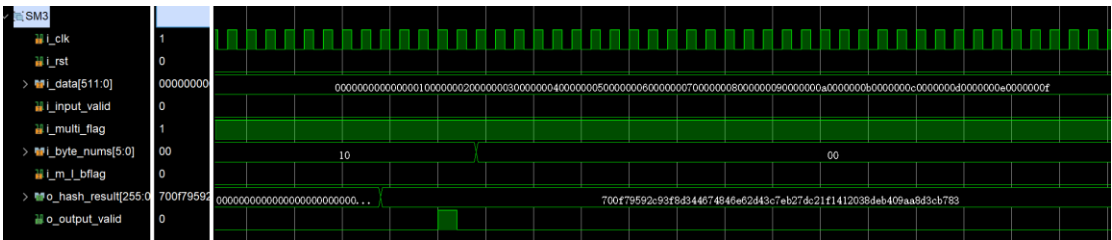


图 14-SM3 模块多组数据运算输入时序-2

输入最后数据分组时序图:

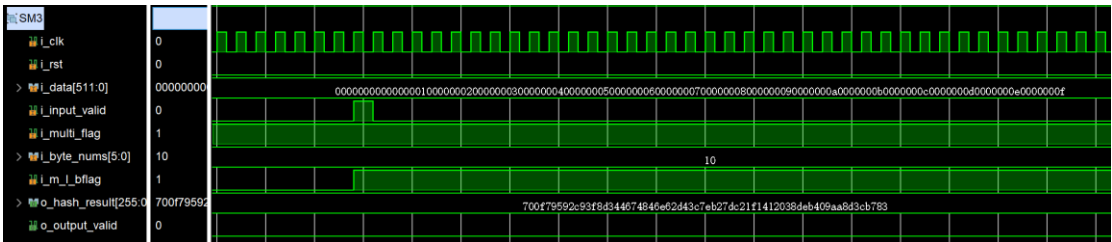


图 15-SM3 模块多组数据运算最后分组输入时序

APB 顶层写地址及数据顺序:

数据及字节长度信息写入地址与单组运算 APB 地址一致; 工作状态写入信息为:
非最后数据分组写入:

写入序号	地址(12 位十六进制)	数据
18	200	32'b101

最后数据分组写入:

写入序号	地址(12 位十六进制)	数据
18	200	32'b111

运算完成清零状态：

写入序号	地址(12 位十六进制)	数据
1	200	32'b000

结果读取 APB 地址：

读取 APB 地址与单租运算 APB 地址一致。

2.1.2 SM4 模块

SM4 IP 模块支持的工作模式分为两层，顶层为 ECB 与 CBC 模式选择，第二层为分为密钥初始化和数据运算两种模式。ECB 与 CBC 模式选择在密钥初始化阶段完成，后续工作中需要保持 ECB/CBC 模式选择位。

以下时序工作说明中 i_sm4_mode 共有 5 位，最高位为 ECB 与 CBC 模式选择，为 1 表示选择 CBC 模式，0 表示 ECB 模式，从密钥初始化开始需要一直保持，该节内容以下忽略该位，所有关于 i_sm4_mode 的说明仅涉及低四位。

i_key_num 接口的高 7 位为 CBC 模式下的初始 IV 值在密钥存储区中的存储地址，需在模块初始化时同步输入，以下内容中忽略该信号高 7 位。

密钥初始化：

密钥初始化完成状态可通过 APB 读取相关寄存器查看。初始化所需时钟周期数为 168。

其支持的初始化模式有

1. 自定义密钥初始化，可向 IP 传输外部密钥并操控其初始化；
2. 使用密钥生成模块生成随机密钥，生成后自动存储在可配置编号参数的（除伪地址陷阱 32、95、127 外的 0-128）安全密钥存储区中，密钥生成后同时会自动送入 SM4 运算模块进行初始化；
3. 使用安全密钥存储区中特定编号的密钥进行初始化。

加解密运算：

可通过配置 APB 状态寄存器控制工作模式决定加密或解密，单次运算所需周期数为 196 个时钟周期。

详细工作时序：

2.1.2.1 自定义密钥初始化

参数：

密钥 i_init_Key 127bits 大端模式
SM4 工作模式选择 i_sm4_mode 4bits （全部为零）

时序图如下：

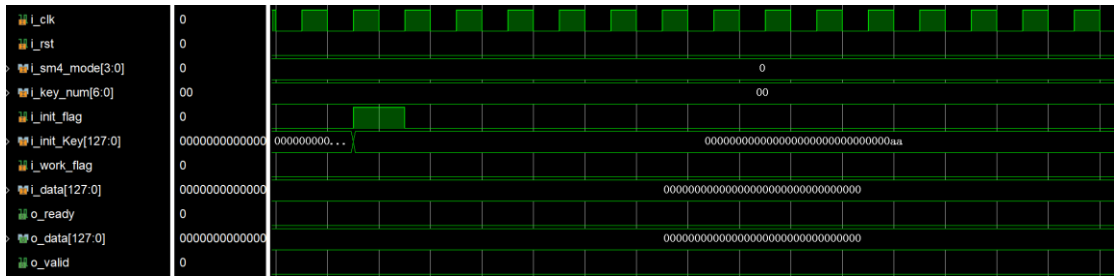


图 16-SM4 模块自定义密钥初始化输入时序

等待 o_ready 信号拉高后即完成初始化，可进行其他操作，所需时钟周期数为 168cycles。

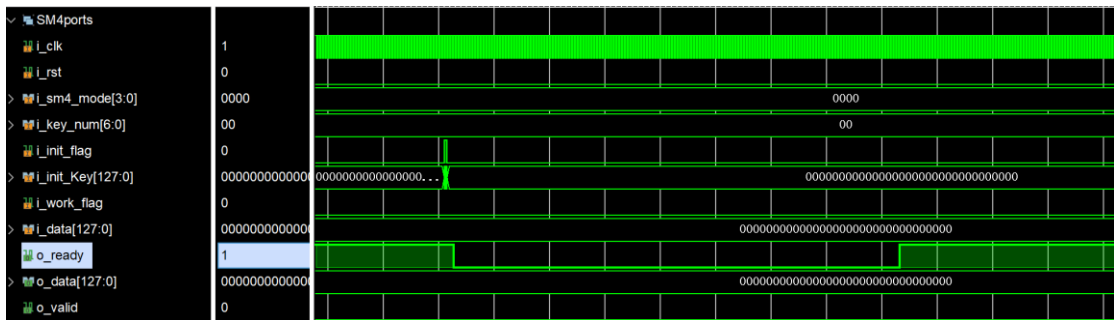


图 17-SM4 模块自定义密钥初始化完成输出时序

APB 顶层写地址及数据顺序：

写入序号	地址（12'hex）	数据
1	14C	4'b0000
2	138	key[31:0]
3	13C	key[63:32]
4	140	key[95:64]
5	144	key[127:96]
6	148	32'b1

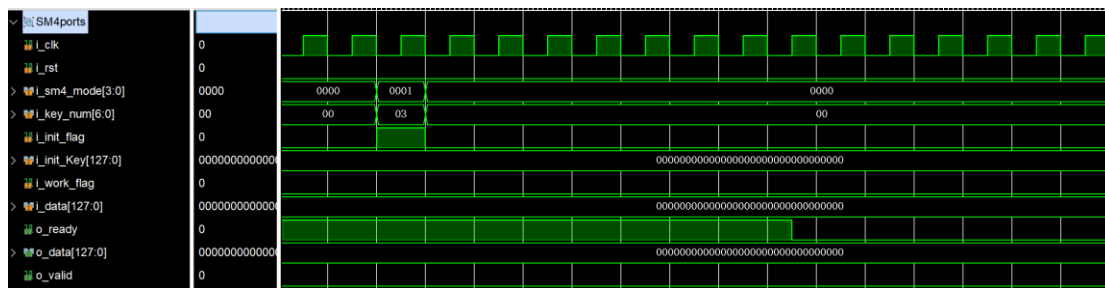
2.1.2.2 使用随机密钥生成模块进行初始化

参数

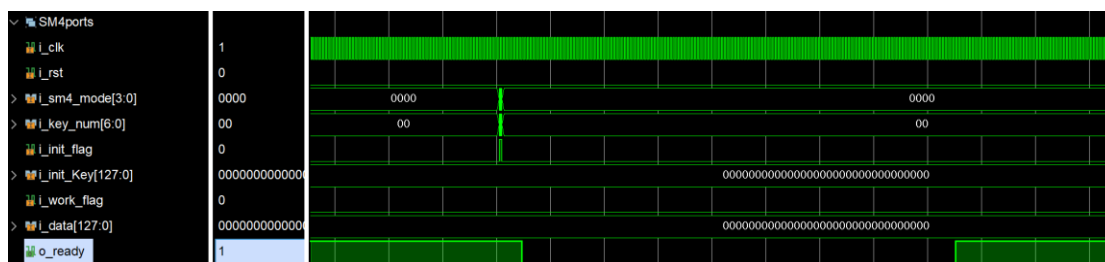
密钥存储区编号 i_key_num 7bits （可选择 0-128 范围内除 32、95、127 外任意数字）

SM4 工作模式选择 i_sm4_mode 4bits （拉高第一位）

时序图如下：



等待 o_ready 信号拉高后即完成初始化，可进行其他操作，所需时钟周期数为 168cycles。



APB 顶层写地址及数据顺序:

写入序号	地址（12'hex）	数据
1	14C	4'b0001
2	150	i_key_num
3	148	32'b1

2.1.2.3 使用安全密钥模块中的密钥进行初始化

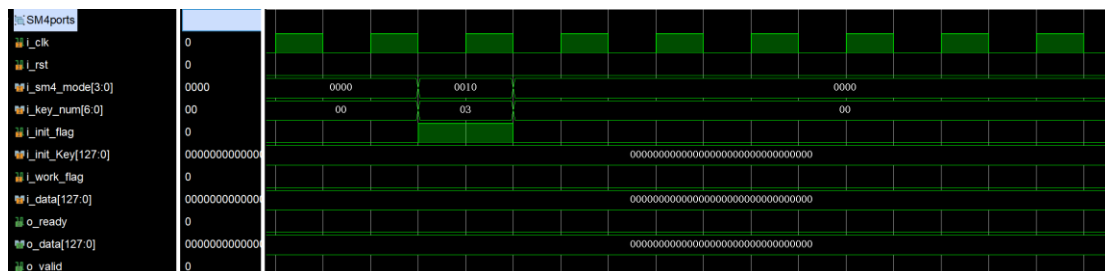
该模式可以操控密钥安全存储模块中某一编号的密钥传入 SM4 运算模块进行初始化;

参数:

秘钥存储区编号 i key num 7bits （可选择 0-128 范围内除 32、95、127 外任意数字）

SM4 工作模式选择 i_sm4_mode 4bits （拉高第二位）

时序图如下:



等待 o_ready 信号拉高后即完成初始化，可进行其他操作，所需时钟周期数为 168cycles。

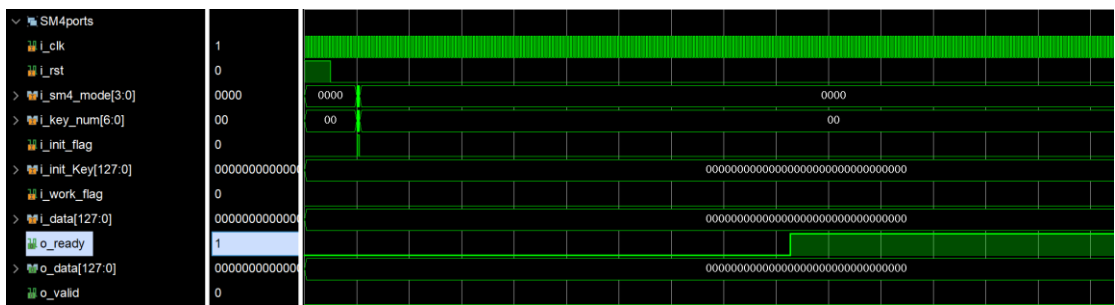


图 21-SM4 模块使用密钥存储模块的密钥初始化完成输出时序

APB 顶层写地址及数据顺序：

写入序号	地址（12'hex）	数据
1	14C	4'b0010
2	150	i_key_num
3	148	32'b1

当 SM4 运算模块完成初始化输出信号 o_ready 为高后,可执行数据加解密任务；

2.1.2.4 数据加密工作模式

输入参数：

128 位待加密数据 i_data 128bits
 SM4 工作模式选择 i_sm4_mode 4bits

输出：

加密数据 o_data 128bits

时序图如下：

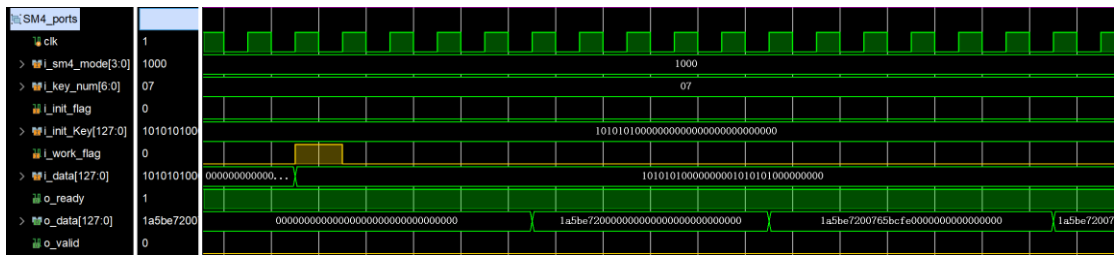


图 22-SM4 模块数据加密输入时序

470 个时钟周期后，o_valid 信号拉高，可读取输出结果数据

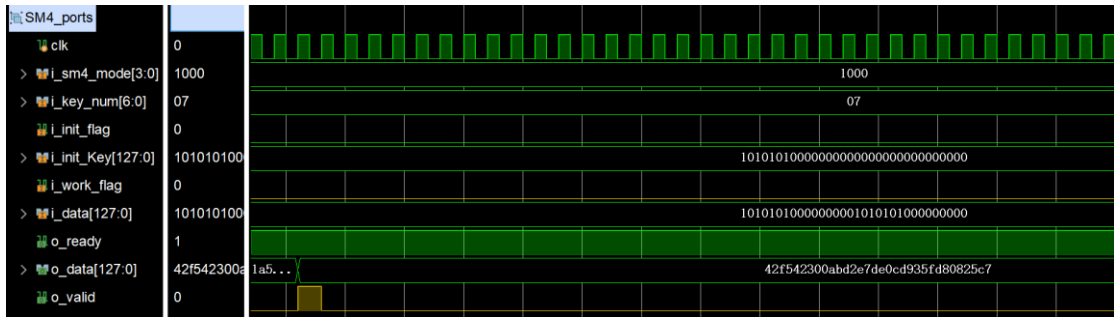


图 23-SM4 模块数据加密完成结果输出时序

APB 顶层写地址及数据顺序：

写入序号	地址（12'hex）	数据
1	14C	4'b1000
2	104	i_data [31:0]
3	108	i_data [63:32]
4	10C	i_data [95:64]
5	110	i_data [127:96]
6	114	32'b1

2.1.2.5 数据解密工作模式

输入参数：

128 位待加密数据 i_data 128bits 大端模式
SM4 工作模式选择 i_sm4_mode 4bits 第三位拉高

输出：

加密数据 o_data 128bits

时序图如下：

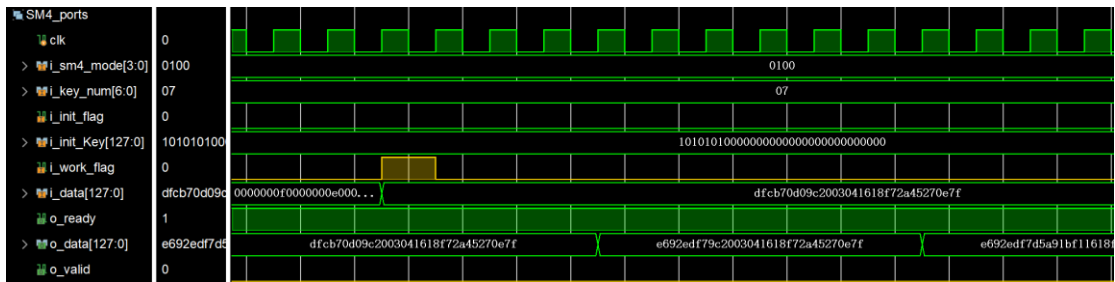


图 24-SM4 模块数据解密输入时序

470 个时钟周期后，o_valid 信号拉高，可读取输出结果数据

2.2.3 安全防护

IP 安全防护设计：

一、

为防止密钥被物理攻击手段窃取，SM4 模块密钥存储区采用分布式 RAM 设计。分布式 RAM 通过 LUT 资源在芯片内分散布局，密钥数据被离散存储于多个独立的 LUT 单元中，而非集中在块 RAM 的连续存储区域。这一特性可显著增加攻击者通过物理分析（如光学探测、电子束成像）定位完整密钥存储位置的难度，从而降低侧信道攻击（如功耗分析、电磁辐射分析）的成功率。

二、

此外，密钥存储模块设置了 ram_shield 信号组（4 个固定值的 128 位常量），这些信号不参与实际存储，但可干扰功耗分析等侧信道攻击，进一步增强了抗侧信道能力。

2.3 接口定义及 APB 协议地址

APB 顶层按照 APB 协议规范设置，地址采用十二位，SM3 与 SM4 的中断单独输出，传输至 SOC 不同状态位，可通过软件层进行读取操作。

2.3.1 APB 顶层接口列表如下

信号名称	方向	位宽	描述
io_apb_PADDR	Input	12 bits	APB 总线地址信号
io_apb_PSEL	Input	1 bit	APB 从机选择信号（高有效）
io_apb_PENABLE	Input	1 bit	APB 传输使能信号（高有效）
io_apb_PREADY	Output	1 bit	APB 从机准备好信号（高有效）
io_apb_PWRITE	Input	1 bit	APB 写使能信号（高为写，低为读）
io_apb_PWDATA	Input	32 bits	APB 写数据总线
io_apb_PRDATA	Output	32 bits	APB 读数据总线
io_apb_PSLVERROR	Output	1 bit	APB 从机错误指示信号（高有效）
io_SM3_interrupt	Output	1 bit	SM3 模块中断信号（高有效）
io_SM4_interrupt	Output	1 bit	SM4 模块中断信号（高有效）
io_mainClk	Input	1 bit	系统主时钟信号
resetCtrl_systemReset	Input	1 bit	系统复位信号（高有效）

2.3.2 APB 顶层模块 APB 协议地址总说明列表

地址（12'h）	访问类型	对应信号/寄存器描述
0x10	读	读取 SM4 就绪状态（sm4_ready_reg）
0x100	读/写	SM4 状态寄存器（SM4_status[1:0]）：读为状态值，写为设置状态
0x104	写	初始数据寄存器 0（ri_initial_data_r0）
0x108	写	初始数据寄存器 1（ri_initial_data_r1）

0x10c	写	初始数据寄存器 2 (ri_initial_data_r2)
0x110	写	初始数据寄存器 3 (ri_initial_data_r3)
0x114	写	控制 SM4 工作标志 (ri_work_flag 及 r_ri_work_flag)
0x118	读	SM4 加密数据 (w_encrypt_data[127:96], 高 32 位)
0x11c	读	SM4 加密数据 (w_encrypt_data[95:64], 次高 32 位)
0x120	读	SM4 加密数据 (w_encrypt_data[63:32], 次低 32 位)
0x124	读	SM4 加密数据 (w_encrypt_data[31:0], 低 32 位)
0x128	读	SM4 解密数据 (w_decrypt_data[127:96], 高 32 位)
0x12c	读	SM4 解密数据 (w_decrypt_data[95:64], 次高 32 位)
0x130	读	SM4 解密数据 (w_decrypt_data[63:32], 次低 32 位)
0x134	读	SM4 解密数据 (w_decrypt_data[31:0], 低 32 位)
0x138	写	初始密钥寄存器 0 (ri_initial_Key_r0)
0x13c	写	初始密钥寄存器 1 (ri_initial_Key_r1)
0x140	写	初始密钥寄存器 2 (ri_initial_Key_r2)
0x144	写	初始密钥寄存器 3 (ri_initial_Key_r3)
0x148	写	设置 SM4 初始密钥(ri_init_Key)及初始化标志(ri_init_flag)
0x14c	写	SM4 模式寄存器 (ri_sm4_mode[4:0])
0x150	写	密钥编号寄存器 (ri_key_num[13:0])
0x200	写	SM3 状态寄存器 (SM3_status_reg[2:0])
0x204	写	SM3 输入数据寄存器 0 (SM3_datain0_reg)
0x208	写	SM3 输入数据寄存器 1 (SM3_datain1_reg)
0x20c	写	SM3 输入数据寄存器 2 (SM3_datain2_reg)
0x210	写	SM3 输入数据寄存器 3 (SM3_datain3_reg)
0x214	写	SM3 输入数据寄存器 4 (SM3_datain4_reg)
0x218	写	SM3 输入数据寄存器 5 (SM3_datain5_reg)
0x21c	写	SM3 输入数据寄存器 6 (SM3_datain6_reg)
0x220	写	SM3 输入数据寄存器 7 (SM3_datain7_reg)
0x224	写	SM3 输入数据寄存器 8 (SM3_datain8_reg)
0x228	写	SM3 输入数据寄存器 9 (SM3_datain9_reg)
0x22c	写	SM3 输入数据寄存器 A (SM3_dataina_reg)
0x230	写	SM3 输入数据寄存器 B (SM3_dataainb_reg)
0x234	写	SM3 输入数据寄存器 C (SM3_dataainc_reg)
0x238	写	SM3 输入数据寄存器 D (SM3_dataaind_reg)
0x23c	写	SM3 数据长度寄存器 0 (SM3_dataalen0_reg)
0x240	写	SM3 数据长度寄存器 1 (SM3_dataalen1_reg)
0x244	写	SM3 字节数寄存器 (SM3_byte_nums_reg[5:0])
0x248	读	SM3 哈希结果寄存器 0 (SM3_dataout0_reg, 哈希值[31:0])
0x24c	读	SM3 哈希结果寄存器 1 (SM3_dataout1_reg, 哈希值[63:32])
0x250	读	SM3 哈希结果寄存器 2 (SM3_dataout2_reg, 哈希值[95:64])
0x254	读	SM3 哈希结果寄存器 3 (SM3_dataout3_reg, 哈希值[127:96])
0x258	读	SM3 哈希结果寄存器 4 (SM3_dataout4_reg, 哈希值[159:128])

0x25c	读	SM3 哈希结果寄存器 5（SM3_dataout5_reg，哈希值 [191:160]）
0x260	读	SM3 哈希结果寄存器 6（SM3_dataout6_reg，哈希值 [223:192]）
0x264	读	SM3 哈希结果寄存器 7（SM3_dataout7_reg，哈希值 [255:224]）

2.3.3 SM3 模块接口定义（代码文件 sm3_top.sv）

信号名称	方向	位宽	描述
i_clk	Input	1 bit	时钟信号
i_rst	Input	1 bit	复位信号（高有效）
i_data	Input	512 bits	消息数据
i_input_valid	Input	1 bit	输入有效信号
i_multi_flag	Input	1 bit	多分组信号，运算数据大于 512 位时需要拉高
i_byte_nums	Input	6 bits	输入单消息分组字节数
i_m_l_bflag	Input	1 bit	多分组运算时最后分组输入信号，高有效
o_hash_result	Output	256 bits	256 位杂凑值数据输出
o_output_valid	Output	1 bit	256 位杂凑值数据输出有效信号

2.3.4 SM4 模块接口定义（代码文件 SM4_TOP_statemac.sv）

信号名称	方向	位宽	描述
i_clk	Input	1 bit	时钟信号
i_rst	Input	1 bit	复位信号（高有效）
i_sm4_mode	Input	4 bits	4 位模式选择信号，[3:0]中只能有一个信号拉高 高位至低位分别对应工作模式为： 加密 解密 使用密钥模块的密钥进行初始化 生成新随机密钥存入密钥存储模块并初始化加解密模块
i_key_num	Input	7 bits	传输到安全密钥存储区的编号
i_init_flag	Input	1 bit	初始化工作信号
i_init_Key	Input	128 bits	自定义初始化密钥传输信号线
i_work_flag	Input	1 bit	加解密工作信号，与数据同时传输
i_data	Input	128 bits	输入数据信号，用于加密/解密
o_ready	Output	1 bit	加解密模块初始化完成信号

o_data	Output	128 bits	输出数据信号线
o_valid	Output	1 bit	输出数据有效信号

2.4 资源需求

该 IP 经 VIVADO 综合后整体资源消耗如表格所示：

Name	Slice LUTs (303600)	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Slice (75900)	LUT as Logic (303600)	LUT as Memory (130800)	Bonded IOB (600)	BUFGCTRL (32)
APB_TOP_SM3_SM4	76502	104483	8336	4104	29067	70486	6016	85	2
usm3 (sm3_top)	29325	29471	0	0	9088	29325	0	0	0
usm4top (SM4_TOP)	47100	73443	8336	4104	19741	41084	6016	0	0

在该 IP 中，

lice LUTs（查找表）：使用了 76502 个；

Slice Registers（寄存器）：使用了 104483 个；

F7 Muxes（7 输入多路复用器）和 F8 Muxes（8 输入多路复用器）：分别使用了 8336 个和 4104 个；

Slice LUT as Logic（用作逻辑的查找表）：29067 个；

Slice LUT as Memory（用作存储的查找表）：7048 个；

Bonded IOB（输入输出块）：使用了 85 个这些用于与外部引脚的连接；

BUFGCTRL（全局时钟缓冲器）：使用了 2 个，用于时钟信号的控制和分发。

2.5.1 C 语言封装函数及软件调用方式

该 IP 各工作模式已封装为 C 语言函数，可在搭载 APB 协议的 32 位系统上方便调用。所有函数定义与声明分别在"软件测试程序\SM3_SM4 测试\code_prj_not_printresults_fast"路径下的 my_func.c 和.\include\my_func.h 文件中。

在比赛 SOC 中 IP 挂载后其地址定义为 0xbf010000，也可适配其他系统进行调整。封装代码中使用指针变量定义该地址：

```
volatile uint32_t* sm34ip = (volatile uint32_t*)0xbf010000;
```

2.5.2 基础 APB 总线读写函数

IP 中各读写参数的 APB 地址通过指针偏移实现，偏移量即为该 IP 的对应数据寄存器 APB 地址。

写入函数：

```
void ip_apbwrite(uint32_t addr, uint32_t data) {
    sm34ip[addr] = data;
}
```

读取函数：

```
uint32_t ip_apbread(uint32_t addr){
    int data;
    data = sm34ip[addr];
    return data;
}
```

2.5.3 SM3 模块功能函数

单分组消息哈希值计算，
函数定义：

```
void hashcompu (uint32_t result[UINT32_COUNT], uint32_t byte_nums, uint32_t stat){

    sm3_flag = 0;

    uint32_t stat_in = (stat<<1)+1;

    ip_apbwrite(0x81,result[0]);           // 204
    ip_apbwrite(0x82,result[1]);           // 208
    ip_apbwrite(0x83,result[2]);           // 20C
    ip_apbwrite(0x84,result[3]);           // 210
    ip_apbwrite(0x85,result[4]);           // 214
    ip_apbwrite(0x86,result[5]);           // 218
    ip_apbwrite(0x87,result[6]);           // 21C
    ip_apbwrite(0x88,result[7]);           // 220
    ip_apbwrite(0x89,result[8]);           // 224
    ip_apbwrite(0x8A,result[9]);           // 228
    ip_apbwrite(0x8B,result[10]);          // 22C
    ip_apbwrite(0x8C,result[11]);          // 230
    ip_apbwrite(0x8D,result[12]);          // 234
    ip_apbwrite(0x8E,result[13]);          // 238
    ip_apbwrite(0x8F,result[14]);          // 23C
    ip_apbwrite(0x90,result[15]);          // 240
    ip_apbwrite(0x91,byte_nums );          // 244
    ip_apbwrite(0x80,stat_in );            // 200
}
```

输入参数分别为：

数据数组起始值指针 data[];

数据字节长度 byte_nums;

二比特工作状态 stat （高位比特代表 i_multi_flag 输入值，低位比特代表 i_m_l_bflag 输入值）。

长数据（大于 512 比特）的杂凑值计算函数：

```

void long_hash (uint32_t data[],uint32_t byte_lens) //max2^32-1 bytes
{
    fin_hash_mul_flag=0;
    uint32_t a = byte_lens / 16;
    uint32_t b = byte_lens % 16;

    if((a>0)&&(b>0)){
        printf("_");
        for(int i=0; i<a;i++){
            hashcompu(&data[i*16],16,2);
            printf("_");
            while (sm3_flag != 1) {
            }
            printf("_");
        }

        for(int h=0; h<(16-b);h++){
            ip_apbwrite(129+h,0);
        }
        uint32_t nums_0 = 129+(16-b);
        for(int x=0; x<b; x++){
            ip_apbwrite(nums_0+x,data[a*16+x]);
        }
        ip_apbwrite(0x91,b );
        ip_apbwrite(0x80,7 );
        printf("_");
        for(int i=0; i<2;i++){
            printf("_");
            while (sm3_flag != 1) {
            }
            printf("_");
        }
    }

    else if((a>1)&&(b==0)){
        printf("_");
        for(int i=0; i<a-1;i++){
            hashcompu(&data[i*16],16,2);
            printf("_");
            while (sm3_flag != 1) {
            }
            printf("_");
        }
        hashcompu(&data[(a-1)*16],16,3);
        printf("_");
        for(int i=0; i<2;i++){
            printf("_");
            while (sm3_flag != 1) {
            }
            printf("_");
        }
    }

    fin_hash_mul_flag=1;
    ip_apbwrite(0x80,0);
}

```

输入参数：

数组名 `data`；

数组总字节长度 `byte_lens`。

结果读取函数：

```
void hash_out_read(uint32_t hash_result[8]) {  
    hash_result[0] = ip_apbread(0X99);           //248  
    hash_result[1] = ip_apbread(0X98);           //24C  
    hash_result[2] = ip_apbread(0X97);           //250  
    hash_result[3] = ip_apbread(0X96);           //254  
    hash_result[4] = ip_apbread(0X95);           //258  
    hash_result[5] = ip_apbread(0X94);           //25C  
    hash_result[6] = ip_apbread(0X93);           //260  
    hash_result[7] = ip_apbread(0X92);           //264  
}
```

该函数需要传入索引为 8 的数组名（已被定义为全局变量，在中断处理中会自动接收）

2.5.4 SM4 模块功能函数

随机密钥生成：

```
void use_key_in_rom(uint32_t order){  
    ip_apbwrite( 83, 2);  
    ip_apbwrite( 84, order);  
    ip_apbwrite( 82,1)  ;  
}
```

该函数传入参数为密钥存储区编号（地址），密钥生成后会自动写入存储区对应地址，同时使用该密钥初始化 SM4 运算模块；

使用密钥存储区某个编号的密钥初始化 SM4 运算模块：

```
void genkey(uint32_t order){  
    ip_apbwrite( 83, 1);  
    ip_apbwrite( 84, order);  
    ip_apbwrite( 82,1)  ;  
}
```

该函数传入参数同样为密钥存储区编号（地址），即 32 位无符号整数 `order`；

通过外部自定义密钥初始化 SM4 运算模块：

```

void direct_init(uint32_t my_key[]){
    ip_apbwrite( 83, 0);
    ip_apbwrite( 78, my_key[3]);
    ip_apbwrite( 79, my_key[2]);
    ip_apbwrite( 80, my_key[1]);
    ip_apbwrite( 81, my_key[0]);
    ip_apbwrite( 82,1) ;
}

```

该函数传入参数为 4 个 32 位无符号整数数组 my_key;

数据加密函数:

```

void text_encrypt(uint32_t text[]){
    sm4_flag=0;
    ip_apbwrite( 83, 8);
    ip_apbwrite( 65, text[3] );
    ip_apbwrite( 66, text[2] );
    ip_apbwrite( 67, text[1] );
    ip_apbwrite( 68, text[0] );
    ip_apbwrite( 69 ,1 ) ;
    for(int i=0; i<2;i++){
        printf("_");
        while (sm4_flag != 1) {
        }
        printf("_");
    }
}

```

该函数传入参数为 128 位待加密数据，形式为包含 4 个 32 位无符号整数的数组名（高位在前）;

解密函数:

```

void chiptext_de(uint32_t chip[]){
    sm4_flag=0;
    ip_apbwrite( 83 , 4);
    ip_apbwrite( 65 , chip[3] );
    ip_apbwrite( 66 , chip[2] );
    ip_apbwrite( 67 , chip[1] );
    ip_apbwrite( 68 , chip[0] );
    ip_apbwrite( 69 ,1 ) ;
    for(int i=0; i<2;i++){
        printf("_");
        while (sm4_flag != 1) {
        }
        printf("_");
    }
}

```

该函数传入参数为 128 位加密数据，形式为包含 4 个 32 位无符号整数的数组名（高位在前）;

SM4 运算模块初始化完成信号读取：

```
uint32_t ready_read(){
    uint32_t a;
    a=ip_apbread(4);
    return a;
}
```

该函数返回值为 SM4 运算模块的 o_ready 信号，若返回值为 1 表示初始化已完成，可以执行数据加解密任务。

2.5.5 中断控制处理

在 SOC 中增加了两个中断位分别用于处理 SM3 与 SM4 模块完成运算后的结果处理。使用上升沿触发的方式：

在 main.c 函数中首先通过 RegWrite 底层封装函数使能系统中断位的第六第七位。代码如下：

```
RegWrite(0xbf20f004,0x6f); //edge 110_1111
RegWrite(0xbf20f008,0x7f); //pol 111_1111
RegWrite(0xbf20f00c,0x7f); //clr
RegWrite(0xbf20f000,0x7f); //en
```

void HWIO_IntrHandler(void)函数中添加内容：

```
else if((int_state & 0x40)== 0x40){
    SM3_IntrHandler();
}
else if((int_state & 0x20)== 0x20){
    SM4_IntrHandler();
}
```

SM3 模块对应中断号为第七位，SM4 模块对应第六位；

SM3 模块中断处理函数：

```
void SM3_IntrHandler(void){

    hash_out_read( sm3_result);
    sm3_flag=1;
    RegWrite(0xbf20f00c,0x40);
}
```

SM4 模块中断处理函数：

```
void SM4_IntrHandler(void){  
  
    sm4_result_read( sm4_result);  
    sm4_flag=1;  
    RegWrite(0xbf20f00c,0x20);  
  
}
```

为了方便检测到中断后处理加解密结果，定义了全局变量：

```
uint32_t sm3_result[8]
```

```
uint32_t sm4_result[4]
```

```
int      sm3_flag;
```

```
int      sm4_flag;
```

sm3_result 与 sm4_result 分别用于接收 SM3 模块与 SM4 运算模块的运算结果；
sm3_flag 与 sm4_flag 用于在软件层指示 IP 工作状态，若完成运算，且 sm3_result 或 sm4_result 数组接收完数据后，该信号会拉高，便于其他程序调取结果。

3.验证方案

采用“仿真验证+FPGA 验证”双重验证体系，覆盖功能正确性、性能达标性以及实际工作稳定性。

3.1 IP 功能正确性验证

该部分包含国家标准文档数据测试以及海量随机生成数据的测试。

3.1.1 标准数据测试

3.1.1.1 SM3

testbench 代码位于目录 " IP 功能性验证\TB_SM3\1_标准示例\standard_data_test_sm3.sv"下。

对《SM3 密码杂凑算法 GB/T32905—2016》国家标准文档附录中示例 2 的 512 比特消息进行运算测试。消息内容如下图：

A.2 示例 2

A.2.1 512 比特消息

61626364 61626364 61626364 61626364 61626364 61626364 61626364 61626364
61626364 61626364 61626364 61626364 61626364 61626364 61626364 61626364

3.1.1.2 SM4

testbench文件位于路径"IP功能性验证\TB_SM4\标准测试\tb_SM4_APB_standard_test.sv"。

该部分对《SM4 分组密码算法 GB/T32907—2016》国家标准文档附录中两个运算示例进行加密测试。

第一个示例为：

输入明文：01 23 45 67 89 AB CD EF FE DC BA 98 76 54 32 10
输入密钥：01 23 45 67 89 AB CD EF FE DC BA 98 76 54 32 10

第二个示例为：

本附录为 SM4 分组密码算法使用固定的加密密钥，对一组明文反复加密 1 000 000 次的运算示例。
输入明文：01 23 45 67 89 AB CD EF FE DC BA 98 76 54 32 10
输入密钥：01 23 45 67 89 AB CD EF FE DC BA 98 76 54 32 10
输出密文：59 52 98 C7 C6 FD 27 1F 04 02 F8 04 C3 3D 3F 66

3.1.2 足量数据稳定性测试

通过海量数据测试的方式验证设计功能正确性以及工作稳定性。步骤如下：

首先在 testbench 代码中调用\$urandom()函数生成随机数据，然后例化 IP 模块，通过任务调用形式对数据进行连续加解密测试，直到到达指定数据量。将加解密运算结果与原始数据保存为 txt 文档，在后续步骤中与 python 标准库对相同数据的加解密结果一致性比对。

3.1.2.1 SM3

代码文件位于路径： " IP 功能性验证\TB_SM3\足量测试\two_thousands_test.sv"下；
分别测试一千条 256 位和 512 位的随机数据。

3.1.2.2 SM4

代码文件位于路径： " IP 功能性验证\TB_SM4\tb_SM4_APB_amount_test.sv"下

分 20 次共测试两千条 128 位随机数据，每一次所使用密钥亦由系统函数随机生成。测试结果分别存放于 20 个文件夹中，路径为"C:\turn_E\prj\VLSI\JC\report\IP 功能性验证\TB_SM4"。

3.1.3 仿真测试 python 验证代码

该部分对足量数据测试中得到的结果进行正确性校验。

文件路径位于" IP 功能性验证\py_test_all "。

该目录下文件说明如下：

“photo”为测试部分截图。

“SM3_DATA_PY” 为 SM3 模块 testbench 测试所使用的随机数据的 python 标准库运算结果

“SM3_DATA_TB” 为 SM3 模块 testbench 测试结果以及 systemverilog 代码

“SM4_DATA_PY” 为 SM4 模块 testbench 测试所使用的随机数据的 python 标准库运算结果

“SM4_DATA_TB” 为 SM3 模块 testbench 测试结果以及 systemverilog 代码，其中每个编号文件夹中的 chip_file.txt 为数据加密后结果，de_chip_file.txt 为对加密结果进行解密后的结果，应与未加密随机数据 text_file.txt 文件中的内容（除第一行为密钥外）完全一致。

“compare_console_output.txt” 为正确结果与 IP 运算结果的一致性对比代码的结果输出。

“compare_correct.py”为完成正确结果与 IP 运算结果的一致性对比代码。

“sm3_hash_256bits.py”为完成一千条 256 位数据 SM3 运算的 python 代码。

“sm3_hash512bits.py” 为完成一千条 512 位数据 SM3 运算的 python 代码。

“sm4_lib_encrypt.py” 为完成 tb_SM4_APB_amount_test.sv 文件生成的随机数据进行标准库加密的 python 代码。

SM3 与 SM4 加解密代码使用 gmssl 依赖库实现，gmssl 库是国内广泛使用的国密算法库（由中国密码学会支持）具备权威性。

验证流程如下：

- 1.调用库函数对 testbench 中生成的随机数据进行加密运算；
- 2.得到结果通过 python 代码与仿真 IP 生成结果进行一致性比对。

3.2 FPGA 验证

方案：对 SM3 模块与 SM4 模块分别编写测试一千条数据的运算，将结果与 Python 调用标准库对相同数据的加密结果对比，若结果一致则通过测试。对 SM3 模块另外运行十组长数据（1kb）测试，验证长数据验签功能。

3.2.1 SM3

1.运算数据使用足量测试中生成的一千条随机数据，运算结果应与仿真和 python 运算结果一致。

该部分测试代码及数据位于路径" 软件测试程序\SM3_SM4 测试\SM3press"下。

2.运行十条长度为 1kb 的随机数据哈希值运算，将结果与 Python 运算结果进行比对。

该部分测试代码及数据位于路径"软件测试程序\SM3_SM4 测试\ten_batches_1kb_test"下。

3.2.2 SM4

先通过 `testbench` 代码生成随机密钥和一千条随机数据，编译将该数据通过软件层函数调用 IP 对数据进行加密运算的程序传入 FPGA 平台进行测试，保存结果与 `python` 代码对同样数据的加密结果进行对比，若一致则通过测试。

该部分测试代码及数据位于路径"软件测试程序\SM3 SM4 测试\SM4press"下。

3.2.3 FPGA 测试代码

该部分顶层目录为"软件测试程序\SM3_SM4 测试"。

子目录及内容说明:

子目录	说明
ten_batches_1kb_test	10 组 1kb 数据 sm3 测试
SM4press	SM4 模块 FPGA 海量数据测试
SM3press	SM3 模块 FPGA 海量数据测试
code_prj_printresults_veri	IP 总体功能覆盖测试
code_prj_not_printresults_fast	IP 软件封装无调试信息输出最简版本

各子目录内容结构:

ten_batches_1kb_test

```
|__1
| |__ hex_data_py.txt //16 进制 1kb 长度数据
| |__ right_result.png //正确哈希值运算结果
| |__ uint32_array_py.txt //数组形式 1kb 数据,用于程序调用
|
|__2 //第二组 1kb 数据，与"1"目录结构相同
.
.
.
```

__ 10	//第十组 1kb 数据，与"1"目录结构相同
__ code_project	//软件程序工程，核心文件为 main.c,data.h,data.c
__ test_result1.png	//FPGA 平台运行结果图 1
__ test_result2.png	//FPGA 平台运行结果图 2
__ hash_long_data.py	//长数据哈希值运算 python 代码
__ sm3_1kb_data.py	//可配置长度参数长数据生成代码

SM4press

__ 1	//testbench 平台生成的一千条随机数据
__ 2	//第二份目录"1"同性质内容
__ 3	//第三份目录"1"同性质内容
__ a_SM3_SM4_press_on_FPGA	//软件程序工程
__ gen_alock.py	//生成调用一千次调用 SM4 加密函数的 python 代码
__ trans_code.py	//将 16 进制数据文件"text_file.txt"生成一千个 C 语言
	//...数组定义的 python 转换代码
__ text_file.txt	//第一份 testbench 平台仿真数据中的明文文件（第
	//...一行为密钥）
__ sm4_data.h	//trans_code.py 生成的数据文件
__ sm4_encrypt_calls.c	//gen_alock.py 生成的调用 C 代码

SM3press

__ correct_result	
	__sm3_results512.txt //text_data_512.txt 中的数据正确哈希值运算结果
__ a_SM3_SM4_press_on_FPGA	//软件程序工程
__ gen_alock.py	//生成调用一千次调用 SM3 运算的 python 代码
__ trans_code.py	//将 16 进制数据文件"text_data_512.txt"每行数据生
	//...成 C 语言数组定义的 python 转换代码
__ text_data_512.txt	//一千条 512bits 数据文本
__ sm3_data.h	//trans_code.py 生成的数据文件
__ long_hash_calls.c	//gen_alock.py 生成的调用 C 代码

code_prj_printresults_veri

__ code_prj	//相关程序代码
__ photo	//测试结果截图

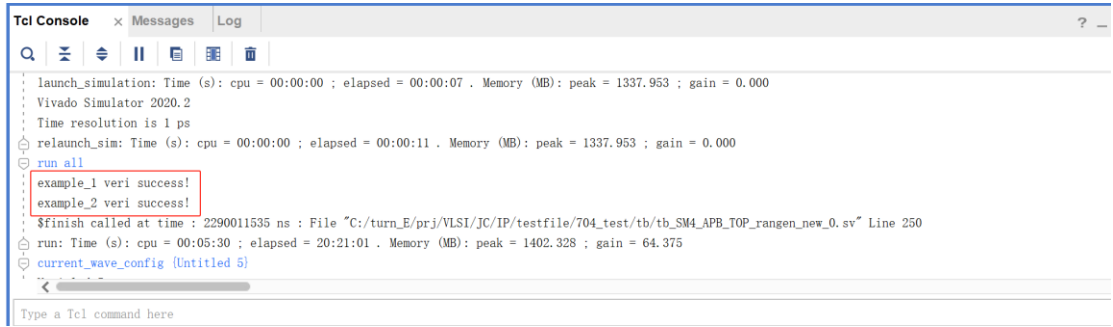
code_prj_not_printresults_fast

__ code_prj	//相关程序代码
-------------	----------

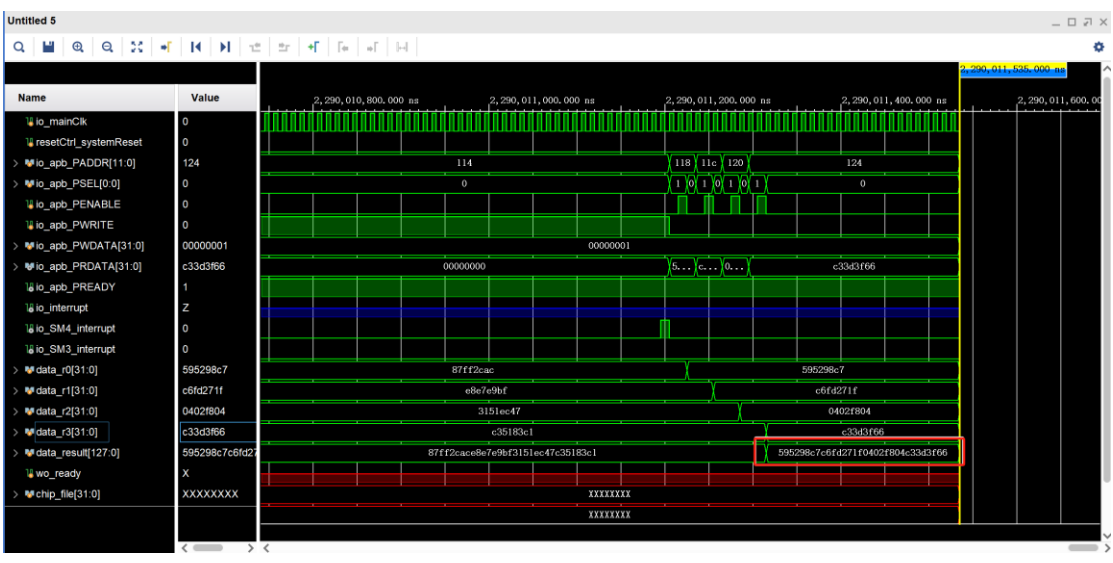
[illegible]

4.1.2 SM4 结果

控制台输出验证成功信号



加密结果波形图



测试核心代码部分:

```
direct_init(128'h0123456789ABCDEFFEDCBA9876543210);
#4000
text_write_encrypt(128'h0123456789ABCDEFFEDCBA9876543210);

if(data_result==128'h681EDF34D206965E86B3E94F536E4246)begin
    $display("example_1 veri success!");
end else begin
    $display("example_1 veri failed!");
end
```

-第一个标准数据加密

```
//example 2 encrypt 1 000 000 times

#100
direct_init(128'h0123456789ABCDEFEDCBA9876543210);
#4000
// en the first time
text_write_encrypt(128'h0123456789ABCDEFEDCBA9876543210);

//remaining 999_999

for(i=1;i<1000000;i=i+1)begin
    text_write_encrypt(data_result);
end

#100
if(data_result==128'h595298C7C6FD271F0402F804C33D3F66)begin
    $display("example_2 veri success!");
end else begin
    $display("example_2 veri failed!");
end
end
```

-1000000 次加密运算

4.2 足量数据测试结果

测试数据位于路径”IP 功能性验证\py_test_all”下：
验证方案中的所有一致性对比结果输出均为内容一致，通过了功能正确性验证。部分结果如图：

```
===== 比对总结 =====
总行数 (以较长文件为准) : 1000
一致的行数: 1000
不一致的行数: 0
行数差异: 0个文件存在多余行

===== 比对总结 =====
总行数 (以较长文件为准) : 1000
一致的行数: 1000
不一致的行数: 0
行数差异: 0个文件存在多余行

===== 比对总结 =====
总行数 (以较长文件为准) : 100
一致的行数: 100
不一致的行数: 0
行数差异: 0个文件存在多余行

===== 比对总结 =====
总行数 (以较长文件为准) : 100
一致的行数: 100
不一致的行数: 0
行数差异: 0个文件存在多余行

===== 比对总结 =====
总行数 (以较长文件为准) : 100
一致的行数: 100
不一致的行数: 0
行数差异: 0个文件存在多余行

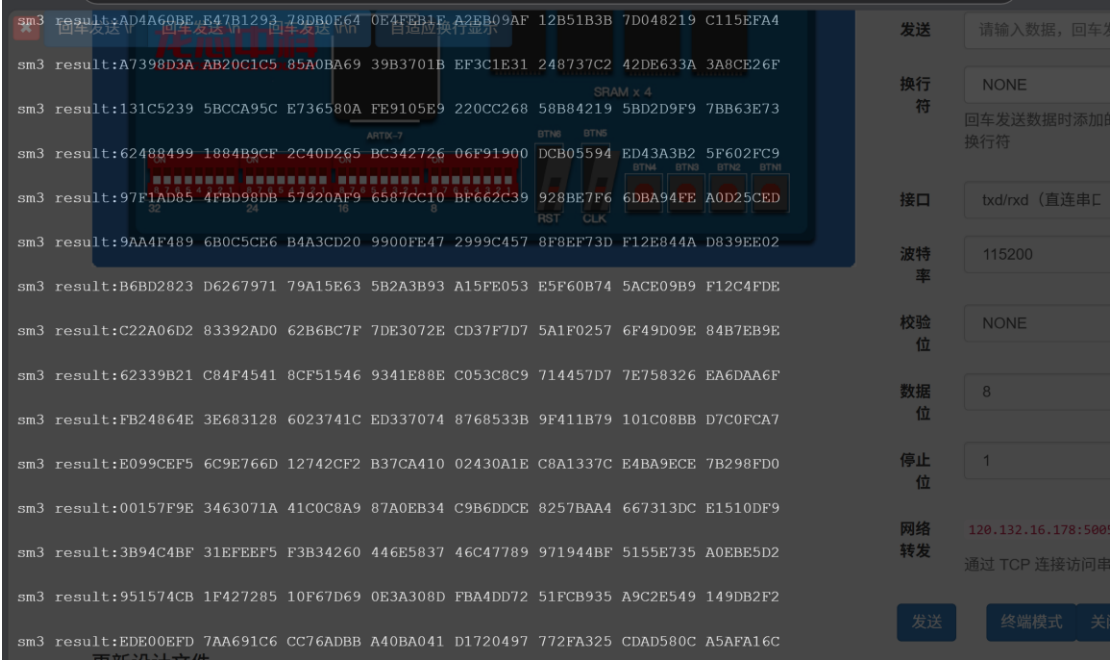
===== 比对总结 =====
总行数 (以较长文件为准) : 100
一致的行数: 100
不一致的行数: 0
行数差异: 0个文件存在多余行

===== 比对总结 =====
总行数 (以较长文件为准) : 100
一致的行数: 100
不一致的行数: 0
行数差异: 0个文件存在多余行
```

4.3 FPGA 测试结果

SM3、SM4 海量数据测试：

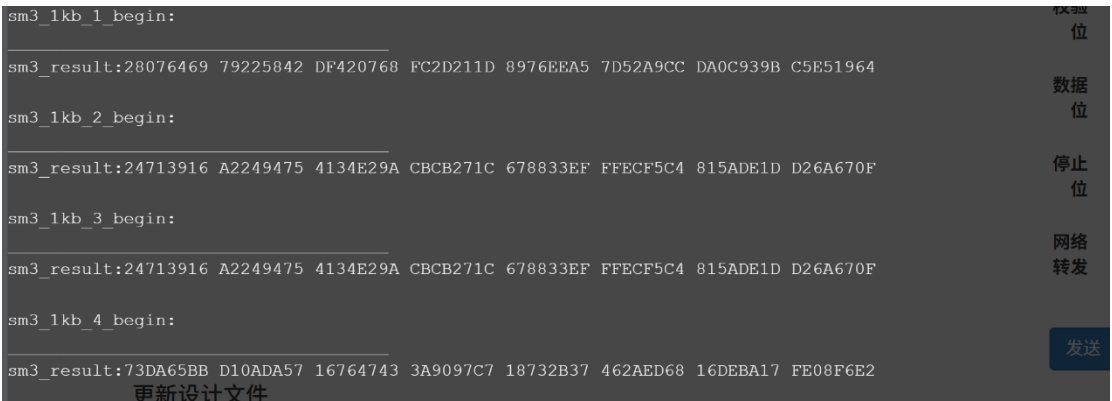
FPGA 测试部分结果如图所示



由于 FPGA 平台终端打印篇幅限制，未能使用 python 代码进行完整的结果检验，经人工检查比对，未发现错误运算数据。

SM3 模块长数据哈希值运算测试：

FPGA 平台输出如图：



```
sm3_1kb_4_begin:
sm3_result:73DA55BB D10ADA57 16764743 3A9097C7 18732B37 462AED68 16DEBA17 FE08F6E2

sm3_1kb_5_begin:
sm3_result:B5F8F349 9C451418 C370FBA8 0D1AFE84 93B131E4 C42A2278 7E72F013 080C5141

sm3_1kb_6_begin:
sm3_result:D7DE1F73 FC3FDC38 2D857605 7AE08382 1258824B A42434A1 C9724BA7 FA9886C7

sm3_1kb_7_begin:
i
sm3_result:E4894608 4140A736 2A8EAB0A F61ED323 475EFC6C 5565CC6C 9AD2F294 13A86EF7

sm3_1kb_8_begin:
sm3_result:EC095AB4 C33AF1AE D0942543 02EDFC2C BA35F6CD 92FCD776 0A5DF550 E38E648E

sm3_1kb_9_begin:
sm3_result:14949C77 07DFE09E 19B98871 AE0F5F6C 49225814 09719B5F 8F4C543F 5AEB79D3

sm3_1kb_10_begin:
sm3_result:74ED0B34 16F09577 0FD28B67 AC3CFF59 0BFF3B8C B63F4EDA 066760C4 C6FDDE62
```

部分测试数据：

```
hex_data_py.txt
文件 编辑 查看
H1  B I  A
427EF06E5E77265ACA18D2F94424349A3FF6C6B1D98C698110ACBF799E105C4D071E7D8583BBE47D93E619072CAE361EBAB
875AAA01717F588F907CC2D560224B5480130033993A130F390ED97C51D2F1365AFE90D2A46A39BB3B57800E04FDFE14CF7
1D215D5A9C0B9A46194BE15B8D9F029AC14DBF2FE57FDCD639A388410CBDF33684F4F4E8B185F0F7F8928924A88B8137F40
9F84AF09214C6DCEFAA97E58BE500242330F57F29301FB4EB3B5AA563E95FB2F8312E5A7E5D1DBBC26E74DC9ABAC9B8FF63
9E216DB164810E1D05F064B67E36510AF04BC60E706BFD13084689834AF8A11229EBF6F7CF8D28FF17EAF4FDD9968B141C8A
58DA6E9287D7F26928F04E899DDF5BC574AB6E35E4F18C7EED020BB8A5488D4E5E24D264664D31817EDD6201FD63564FBE
BF465921045DA7304AD37E7F2787E132994A8A0DDEB87D1DE0B080088745B579B8031D06A12CDCCC33892E021C583535EB
E0AAC61BB03688F1854CBB62C3A478575DF7A793A9AE7550EB9248AB427FEB39D4405F28E86C741BE30D97586F4036B70
1B52012AA995BB2AC990F3E2482D2B48FF63591A891AA05EA7EBC0B3F8E3485B63AFDCADD1CFB4EB33D55608165B2E70C1
67B6CA7768C40D6EA8516567DC15AFD8C093B28CA7F9F8DC9FDA0FE1B6386F21CFA64535446EDDBC4CA7AB143F1CAC8
7C2E673C28BB1125B906F62C55BA9CD65DD4C5FCDE33985E9391E549A3BCC09D0B94A18FFBC508B25E0F924A904C367F65
C365960A1A8C44529506A9DD7660495B9B0B5467A00ABDD0D4F366E81AAD299A24C42A9F544687833C71241DC35D6CC5
79AB4DE867FEE9150D7F92D49C1D4589A201AB5B7FB2B6B21D50674C44823E3A613E2E78D72E424C294D03A41CD72D6E8C
658A56462E92CF96D253777581D4E48F4B8DF7A4C4BF3768E2571AC865881968AA871EAA3A2079788F77689C3F4426CEA
5D753455A0292505D35E115C69E7F8295EF69BFB0A415221B78D22F10B41310F7BC43E73B52E78CBE2AFED5DB3FEAF93D0F2
37FC21FBD6D52AE58847D71F57B715C770B77388619EEE8918AB5D1D89006A9E5006DFAC442A65AF6672A2619A922592F80
F8EE2A70ECFB9455B3F8964D4D79F17C08B0EE69D3DF2785F6B09A8462EA1F7E3F05370B7841CE0A3DB1AD4ED5AB5198146
41E1FC269A1EF4BFD06FC3B5DB1AEA6B5A8E5AFF80736FC7838D8C55F09605C6080FD9B41CCEFE76C42F3ACF405ECF36AE
279DC0DD42B387FA72F95C7C8C21BE527E6F6D88D8EBB02B934DF8CC70ED36BED4DF535457E8489A9D76762B8E17BF8079
7A2F55DC1F35EC9743C0AB25D690BEF79BDBB002691115BE6E82B73A8D949BE9CB421FE547AA8FE6ECBED99AB4DC4DC7A
99DDE0531C860535A8964EB34FBCD1294AD087BBB76030B1C582A517CCF181AADFE601D59F03F672
```

正确杂凑值：

```
E:\ANACONDA\envs\YOLO_6GPU\python.exe C:\turn_E\prj\VLSI\JC\report\FPGA\IPress\hash_long_data.py
SM3加密结果（64位16进制）：
24713916a22494754134e29acbc271c678833efffecf5c4815ade1dd26a670f
```

经过比对，所有 1kb 测试数据 FPGA 运算结果均为正确值，通过 SM3 模块长数据运算测

试。

4.4 代码覆盖率及 FPGA 时序报告

通过整合所有模块测试代码，得到覆盖率报告结果如图

Code Coverage Summary						
Total Files	Total Modules	Total Instances	Statement Coverage Score	Branch Coverage Score	Condition Coverage Score	Toggle Coverage Score
14	14	491	98.7607	97.7933	82.8283	34.31

图 26 覆盖率报告

本次代码覆盖率报告显示在语句和分支覆盖方面表现较好，条件覆盖率相对较低是由于硬件设计中的信号控制原因使得部分条件不可能出现，执行设定功能已完全覆盖，可忽略。

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 1.384 ns	Worst Hold Slack (WHS): 0.009 ns	Worst Pulse Width Slack (WPWS): 7.000 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints:	0
Total Number of Endpoints: 274909	Total Number of Endpoints: 274909	Total Number of Endpoints:	125301
All user specified timing constraints are met.			

图 27 自主 IP Vivado 时序报告

自主 IP 集成于 SOC 后在 VIVADO 平台布局后的时序报告如上，可见总体时序性能良好，所有指定约束均已满足，关键指标裕量充足，无任何时序违规风险，具有高可靠性，体现了良好的设计。